

Escaping Stability-Plasticity Dilemma in Online Continual Learning for Motion Forecasting via Synergetic Memory Rehearsal

Yunlong Lin¹, Chao Lu¹, *Member, IEEE*, Tongshuai Wu, Xiaocong Zhao², *Member, IEEE*, Guodong Du¹, Yanwei Sun, Zirui Li¹, *Member, IEEE*, and Jianwei Gong¹, *Member, IEEE*

Abstract—Deep neural networks (DNNs) have achieved remarkable success in intelligent systems such as autonomous vehicles and robots. However, most DNN-based methods suffer from catastrophic forgetting, where the DNN may fail to maintain its performance in previously learned tasks after adapting to new data. This phenomenon gives rise to a critical tradeoff between *memory stability* of the DNN, that is, the ability to retain learned knowledge, and *learning plasticity*, that is, the capacity to acquire new information effectively. To address such stability–plasticity dilemma, this study proposes a novel CL method named synergetic memory rehearsal (SyReM). SyReM maintains a compact memory buffer to represent learned knowledge. To ensure memory stability, it employs an inequality constraint that limits increments in the average loss over the memory buffer. Synergistically, a selective memory rehearsal mechanism is designed to enhance learning plasticity by selecting samples from the memory buffer that are similar to recently observed data. This selection is based on an online-measured cosine similarity of loss gradients, ensuring targeted memory rehearsal. We validate SyReM under online CL tasks for motion forecasting. Comprehensive experiments on naturalistic driving datasets demonstrate that, compared to non-CL and CL baselines, SyReM significantly mitigates catastrophic forgetting in

past scenarios while improving forecasting accuracy in new ones. The code is available at <https://github.com/BIT-Jack/SyReM>

Index Terms—Artificial neuroplasticity, autonomous driving (AD), catastrophic forgetting, memory rehearsal, motion forecasting, online continual learning (CL).

I. INTRODUCTION

DEEP neural networks (DNNs) have been successfully applied in motion forecasting [1], which is important for safe planning in autonomous driving (AD) [2], [3]. As naturalistic scenarios evolve dynamically, AD systems need to adapt and remember incrementally encountered scenarios throughout their lifespans [4]. To enhance generalization performance across different scenarios, most motion forecasting methods train DNNs using a vast amount of data, trying to cover as many scenarios as possible [5]. However, real-world scenarios are infinite. Simultaneously maintaining massive data from potentially unlimited scenarios overwhelms the finite storage and computation resources of AD systems [6], [7]. In such cases, continuously updating the DNN as new data arrives is a potentially feasible paradigm for DNN-based models to obtain consistently accurate predictions across different scenarios [8]. Nevertheless, DNN-based methods may suffer severe performance degradation in previously learned scenarios after adapting to new ones [9], a phenomenon known as catastrophic forgetting [10]. This phenomenon arises mainly because when learning new scenarios with shifted data distributions, the trainable parameters of the DNN tend to be optimized for the new distribution. Such parameter updates overwrite the knowledge essential for previously learned scenarios [6].

Continual learning (CL) is one of the solutions to address the catastrophic forgetting of DNN [11]. CL aims to enable DNNs to learn sequential tasks with no or limited access to old data, while achieving consistently good performance on all encountered tasks [6]. This goal of CL essentially boils down to resolving the stability–plasticity dilemma: *memory stability* refers to the capability of avoiding catastrophic forgetting on past tasks, and *learning plasticity* denotes the capacity to adapt to new tasks efficiently [12], yet the two compete for the same parameter space in DNN [11]. As illustrated in Fig. 1(a), the DNN updates its parameters to fit data distributions from the new task $\mathcal{T}_c$, which risks eroding the parameter configurations that underpin memory stability. Consequently, catastrophic forgetting happens, leading to poor performance

Received 23 August 2025; revised 10 March 2026 and 12 May 2026; accepted 1 June 2026. This work was supported in part by the Key Technological Breakthrough Projects for Major Industries in Henan Province under Grant 251000240100, in part by the National Natural Science Foundation of China under Grant 52372405, in part by Beijing Natural Science Foundation under Grant L247033, and in part by China Scholarship Council (CSC). (Corresponding authors: Zirui Li; Jianwei Gong.)

Yunlong Lin is with the School of Mechanical Engineering, Beijing Institute of Technology, Beijing 100081, China, and also with the Cyber-Physical Systems Group, School of Computation, Information, and Technology, Technical University of Munich, 85748 Garching, Germany (e-mail: jacklyl.bit@gmail.com).

Chao Lu and Jianwei Gong are with the School of Mechanical Engineering, Beijing Institute of Technology, Beijing 100081, China, and also with Zhengzhou Research Institute, Beijing Institute of Technology, Zhengzhou 450000, China (e-mail: chaolu@bit.edu.cn; gongjianwei@bit.edu.cn).

Tongshuai Wu and Guodong Du are with the School of Mechanical Engineering, Beijing Institute of Technology, Beijing 100081, China (e-mail: wutongshuai84@gmail.com; guodongdu_robbie@163.com).

Xiaocong Zhao is with the Key Laboratory of Road and Traffic Engineering, Ministry of Education, Tongji University, Shanghai 201804, China (e-mail: zhaoxc@tongji.edu.cn).

Yanwei Sun is with Zhengzhou Nissan Automobile Company Ltd., Zhengzhou 450046, China (e-mail: sunyanwei@znnissan.com.cn).

Zirui Li is with the School of Mechanical Engineering, Beijing Institute of Technology, Beijing 100081, China, and also with the School of Mechanical and Aerospace Engineering, Nanyang Technological University, Singapore 639798 (e-mail: zirui.li.work.bit@gmail.com).

Digital Object Identifier 10.1109/TNNLS.2026.3700190

2162-237X © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: CHONGQING UNIVERSITY. Downloaded on August 29, 2026 at 02:31:18 UTC from IEEE Xplore. Restrictions apply.

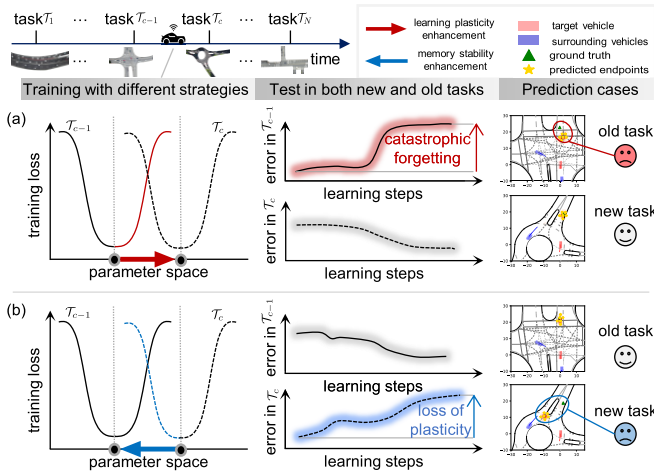


Fig. 1. Stability-plasticity dilemma in CL tasks of motion forecasting. (a) Optimizing for the new task may overwrite parameters that are important to old tasks, leading to catastrophic forgetting in old tasks. (b) Excessive memory stability may restrict the adaptation of DNN to new tasks, causing a loss of learning plasticity.

in old tasks (e.g., task τ_{c-1}). Conversely, rigidly preserving parameters to maintain memory stability would stifle the necessary adjustments for learning plasticity, impeding the model from effectively adapting to new tasks [Fig. 1(b)]. This inherent competition gives rise to the stability-plasticity dilemma.

In motion forecasting, CL tasks are defined as predicting trajectories or behaviors of road users (e.g., vehicles, pedestrians, and cyclists) across sequential scenarios [13], [14]. Corresponding datasets collected from various scenarios arrive as a sequence incrementally. Under the assumption of limited data usage from past tasks, recent studies attempted to alleviate catastrophic forgetting and aimed to minimize the averaged prediction errors in test sets of all encountered tasks [15]. Nevertheless, most of these studies follow the offline CL manner, where the DNN can be trained for any epoch based on the fully available dataset from the current task [16]. However, allocating exclusive time to offline training, a process that imposes heavy demands on large datasets and entails lengthy training cycles, is scarcely viable in AD deployments with stringent real-time requirements [17].

By contrast, online CL trains a DNN using a one-pass data stream [11], which could be a more practical paradigm for motion forecasting in AD [18]. A recent study has empirically shown the poor memory stability of state-of-the-art (SOTA) DNN-based motion forecasting models when learning from a one-pass data stream in the online CL paradigm [19]. On the other hand, recent theoretical findings have demonstrated that DNN-based methods in CL settings gradually lose learning plasticity until no better than a shallow network [20]. Directly applying established CL-based motion forecasting methods might further harm the learning plasticity in the online CL paradigm [12]. As a result, most DNN-based motion forecasting methods still get trapped in the dilemma of trading off maintaining learned knowledge and adapting to new data in an online manner. Escaping such a stability-plasticity dilemma of online CL for motion forecasting is significant but challenging

for the development of high-level AD systems. To our best knowledge, few studies have investigated this research field.

To fill this research gap, this study proposes a novel CL method, termed Synergetic Memory Rehearsal (SyReM), for DNN-based motion forecasting to escape the stability-plasticity dilemma. First, we design a memory buffer that encapsulates knowledge learned from samples of previously encountered tasks, serving as a foundational component in the SyReM. For memory stability, the primary goal is to prevent the erosion of knowledge from prior tasks when the model adapts to new data distributions. Given that DNN parameter updates rely on gradient descent, SyReM employs a gradient projection strategy based on the memory buffer. This strategy dynamically adjusts the gradient descent process within the online CL paradigm, constraining the incremental increase of the averaged loss over the memory buffer. Since the learned knowledge is represented by the buffer, this constraint mitigates the forgetting of historical tasks as the model adapts to new data distributions. Synergistically, an online similarity measurement-based rehearsal strategy is developed for learning plasticity. As the objective is to enhance the adaptive efficiency for new tasks by leveraging relevant prior knowledge, the strategy selects samples from the memory buffer that are most similar to the latest observed samples in the new task and rehearses them to the DNN at each training step. This process facilitates the transfer of useful prior knowledge to boost adaptation to new tasks. As the rehearsed samples are drawn from the memory buffer, which stores samples from old tasks, they do not introduce unnecessary interference with the acquired prior knowledge. This enables SyReM to enhance learning plasticity for new tasks without compromising the memory stability of old tasks. The main contributions of this paper are summarized as follows.

- 1) A novel CL method, SyReM, is proposed to resolve the stability-plasticity dilemma in DNN-based motion forecasting. It guarantees memory stability by constraining increments in the average loss over learned tasks, while synergistically enhancing learning plasticity through a selective memory rehearsal mechanism that leverages prior knowledge.
- 2) Two core mechanisms of SyReM are tailored for the online CL paradigm, where sequential task data arrives as a one-pass stream without multi-epoch training on static task batches. First, an online memory buffer is constructed via reservoir sampling to capture the distribution of learned samples. Second, a selective rehearsal strategy compares cosine similarities of loss gradients between old and new samples, enabling real-time guidance for memory rehearsal at each learning step.
- 3) Comprehensive experiments are conducted using diverse metrics to systematically evaluate memory stability and learning plasticity. Experimental results demonstrate that SyReM outperforms SOTA CL methods in simultaneously enhancing stability and plasticity. Beyond addressing the dilemma, SyReM exhibits superior zero-shot generalization.

The remainder of this article is organized as follows. Section II first introduces the related works to distinguish this study from previous works. Then, online CL tasks for motion forecasting are formulated in Section III. The proposed

method is detailed in Section IV. Next, experimental settings, results, and discussion are presented in Section V, respectively. Finally, the conclusion and future works are summarized in Section VI.

II. RELATED WORK

This work seeks to enable DNN-based motion forecasting models to address the stability–plasticity dilemma in CL. In this section, existing studies on DNN-based motion forecasting are first reviewed to provide the necessary background. Next, the fundamental concepts and mainstream strategies of CL are introduced. Besides, to contextualize the dilemma and further delineate our contributions, related paradigms, including transfer learning (TL) and meta-learning, are briefly surveyed.

A. DNNs-Based Motion Forecasting

DNN-based motion forecasting predicts future trajectories of road users (e.g., vehicles, pedestrians, and cyclists) from their historical motion states and scene context [5]. Early studies used recurrent neural networks such as long short-term memory and gated recurrent unit to learn temporal patterns [21]. To capture spatial features, convolutional neural networks processed high-definition maps or bird’s-eye-view images [1], [22]. Recent studies have leveraged graph neural networks [18], [23], [24] and transformer architectures [25] to enhance interaction reasoning among agents. Meanwhile, diffusion-based generative models have also emerged as a promising direction for motion forecasting, owing to their strong ability in distribution modeling. For example, MoFlow [26] achieves efficient and diverse trajectory prediction using one-step flow matching and implicit maximum likelihood estimation distillation, and DiffWT [27] introduces discrete wavelet transform into diffusion models to capture multiscale motion details for pedestrian trajectory prediction. However, most established DNN-based methods were evaluated using test data that is independent and identically distributed (i.i.d.) with the training data. The struggle with adaptation to out-of-distribution limits their applicability in evolving scenarios.

B. Continual Learning

CL is also termed lifelong learning [4] or incremental learning [11]. CL allows sequential task learning without forgetting past knowledge. Common strategies include regularization-based [28], architecture-based [29], and replay-based [30]. Replay-based strategies are widely used in online CL by adapting to streaming inputs with bounded memory [31]. Replay-based strategies store past examples for memory rehearsal [6]. Nevertheless, most replay-based methods favor memory stability on past tasks at the expense of learning plasticity for new tasks [20].

There are a few paradigms aiming to enhance the learning plasticity for DNNs, including TL [32] and meta-learning [33]. TL usually pretrains models on large source datasets and fine-tunes them on target tasks. Although the learning plasticity can be enhanced by TL with limited data, most TL-based models suffer from domain mismatch and catastrophic forgetting. Meanwhile, TL requires access to target data during training and may not continuously handle streaming scenarios. Alternatively, meta-learning learns task-agnostic

initializations for fast adaptation [33]. However, meta-learning relies on diverse meta-training tasks and exhibits high computational overhead. In summary, although TL and meta-learning have demonstrated progress in improving the learning plasticity of DNNs, these paradigms might fail to reconcile stability with plasticity and lack mechanisms for continuous updating. Trading off memory stability and learning plasticity is a critical trend for improving the practicability of DNN-based motion forecasting.

Recent CL methods for class/task-incremental classification adopt dual or multiple memory-related mechanisms to balance stability and plasticity, for example, sample selection tailored for high-value exemplars [34], sparse coding-based representation regularization [35], and bidirectional representation constraint via dual knowledge bases [36]. However, they rely on explicit task boundaries and task-specific triggers for memory and representation updates, which may not naturally extend to online domain-incremental learning [11]. Although their core designs can address catastrophic forgetting of discrete class labels, they would struggle to address performance degradation arising from spatiotemporal distribution drifts in motion forecasting.

III. PROBLEM FORMULATION

Motion forecasting for AD aims to estimate the future positions or behavior intentions of road users such as vehicles, pedestrians, and cyclists. In this context, target agents (TAs) are defined as road users under prediction. Surrounding agents (SAs) are defined as other road users that can influence TAs through physical interactions or environmental constraints. DNN-based motion forecasting models leverage the historical motion states of both TAs and SAs to achieve spatiotemporally accurate predictions.

Formally, let t_c denote the current time step within the observation horizon t_{obs} . The input to a DNN-based motion forecasting model is represented as $\mathbf{X} = [\mathbf{X}_{t_c - t_{\text{obs}} + 1}, \dots, \mathbf{X}_{t_c}]$, where $\mathbf{X}_t$ contains motion states (e.g., positions and velocities) of the TAs and SAs, and map features (e.g., road boundary and centerline of lanes) at time t . Let f_θ denote the DNN-based model parametrized by θ . Given a forecasting time window t_{pred} , the output $f_\theta(\mathbf{X})$ estimates the future motion states $\mathbf{Y}$ of the target agent at time $t_c + t_{\text{pred}}$.

In the online CL setting, the model f_θ will encounter $N \in \mathbb{Z}^+$ tasks $\{\mathcal{T}_1, \dots, \mathcal{T}_N\}$ sequentially. Let i denote the index of tasks, and let j denote the index of samples. $\forall i \in [1, N]$, each task $\mathcal{T}_i$ contains $N_{\mathcal{T}_i}^{\text{tr}}$ training samples drawn from an i.i.d. distribution $\mathcal{D}_{\mathcal{T}_i}$. Toward the online characteristic [11], the corresponding training samples of all CL tasks arrive as a one-pass data stream

$$\mathcal{S} = \{(\mathbf{X}, \mathbf{Y})_j \sim \mathcal{D}_{\mathcal{T}_i} | 1 \leq i \leq N, 1 \leq j \leq N_{\mathcal{T}_i}^{\text{tr}}\}. \quad (1)$$

We use $\mathcal{T}_c$ to denote the current task that the model is learning, where the subscript $c \in [1, N]$. Then, the past tasks are defined as the set of previously learned tasks $\{\mathcal{T}_i\}_{i=1}^{c-1}$. Since memory stability refers to the ability to retain learned knowledge in past tasks, the training loss of memory stability can be formulated as

$$\mathcal{L}_{\text{stability}} = \sum_{i=1}^{c-1} \mathbb{E}_{(\mathbf{X}, \mathbf{Y}) \sim \mathcal{D}_{\mathcal{T}_i}} (\ell(f_{\theta_i}(\mathbf{X}), \mathbf{Y})) \quad (2)$$

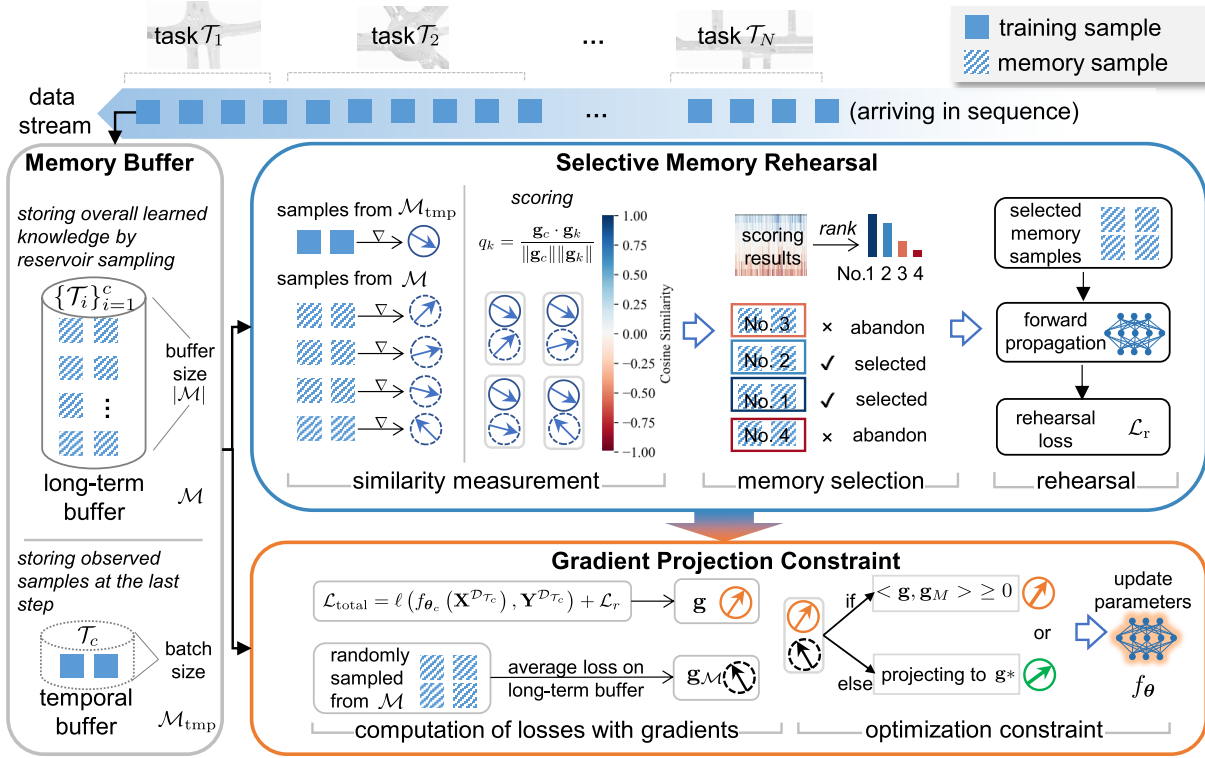


Fig. 2. Proposed SyReM method. The memory buffer contains a long-term buffer $\mathcal{M}$ and a temporal buffer $\mathcal{M}_{tmp}$. First, overall knowledge from the online data stream is maintained in $\mathcal{M}$ via reservoir sampling, and the observed samples at the last learning step are temporarily stored in $\mathcal{M}_{tmp}$. Then, similarity scores are computed by comparing the full-network gradient cosine similarity between $\mathcal{M}_{tmp}$ and candidates from $\mathcal{M}$. Memory samples with the higher scores are selected for the rehearsal, which enhances the learning plasticity. Finally, the gradient projection employs an inequality constraint for the DNN update to further improve memory stability.

where $\ell(\cdot)$ is the loss function, and $\mathbb{E}$ denotes the mathematical expectation. The f_{θ_i} represents the motion forecasting model trained on task $\mathcal{T}_i$. By contrast, learning plasticity refers to the ability of a DNN to learn new tasks. This necessitates utilizing learned knowledge from past tasks to obtain accurate predictions in the current task efficiently. Thus, the loss of learning plasticity is represented as

$$\mathcal{L}_{\text{plasticity}} = \mathbb{E}_{(\mathbf{X}, \mathbf{Y}) \sim \mathcal{D}_{\mathcal{T}_c}} (\ell(f_{\theta_c}(\mathbf{X}), \mathbf{Y})). \quad (3)$$

However, the excessive stability may hinder efficient adaptation to the new task, while over-focusing on the plasticity may aggravate catastrophic forgetting in past tasks. The stability-plasticity dilemma arises from the difficulty of achieving the objectives (2) and (3) simultaneously

$$\theta_{1:c}^* = \arg \min_{\theta_{1:c}} \mathcal{L}_{\text{stability}} + \mathcal{L}_{\text{plasticity}}. \quad (4)$$

Here, we use $\theta_{1:c}$ to denote the parameters of the model that are trained from the first task $\mathcal{T}_1$ to the current task $\mathcal{T}_c$. The aim is to find the optimal parameters $\theta_{1:c}^*$, minimizing both the loss of memory stability and learning plasticity.

IV. SYREM FOR ONLINE CL

This study proposes a method, SyReM, for DNN-based motion forecasting to escape the stability-plasticity dilemma. The overall schematic of SyReM is demonstrated in Fig. 2. First, a memory buffer is designed to partially store learned samples from the online data stream. Then, as a core component of SyReM, the selective memory rehearsal aims at

enhancing learning plasticity by leveraging samples from the memory buffer on a selective basis through a similarity measurement. Combined with the rehearsal loss output by the selective memory rehearsal module, the gradient descent on the total loss is constrained for the guarantee of memory stability via a gradient projection strategy. More details are presented as follows.

A. Memory Buffer for Online Data Stream

The proposed methodology is built on a replay-based CL mechanism, representing learned knowledge by a subset of observed samples. For the representation of knowledge from past tasks, we use a long-term memory buffer $\mathcal{M}$ to dynamically maintain a subset of observed samples from the data stream, called memory samples. Due to the resource efficiency of CL, the amount of memory samples in the long-term buffer, that is, the buffer size denoted as $|\mathcal{M}|$, is required to be far less than the total number of training samples [6]

$$|\mathcal{M}| \ll \sum_{i=1}^N N_{\mathcal{T}_i}^{\text{tr}}. \quad (5)$$

With the limited memory resources, the long-term buffer is designed to approximate the overall distribution of all learned samples. Equiprobable sampling is an effective strategy for obtaining an unbiased estimation of the entire training set. However, in online CL, samples become sequentially available through the data stream defined in (1), where the total length

Algorithm 1 Sampling Strategy for Memory Buffer

Input: The data stream $\mathcal{S}$ with unknown length $|\mathcal{S}| = \sum_{i=1}^N N_{\mathcal{T}_i}^{\text{tr}}$; the k^{th} sample $(\mathbf{X}, \mathbf{Y})_k$ from $\mathcal{S}$; the long-term buffer $\mathcal{M}$ with the buffer size $|\mathcal{M}|$; the temporal buffer $\mathcal{M}_{\text{tmp}}$.

Output: The updated $\mathcal{M}$, where the r^{th} sample stored in $\mathcal{M}$ is denoted as $\mathcal{M}(r)$; the updated temporal buffer $\mathcal{M}_{\text{tmp}}$

```

1  $\mathcal{M} \leftarrow \emptyset$ ;  $\triangleright$  Initialization of the long-term buffer.
2  $\mathcal{M}_{\text{tmp}} \leftarrow \emptyset$ ;  $\triangleright$  Initialization of the temporal buffer.
3 for  $k$  in  $\text{range}(1, |\mathcal{S}|)$  do
4    $\mathcal{M}_{\text{tmp}} \leftarrow (\mathbf{X}, \mathbf{Y})_k$ ;  $\triangleright$  Update the temporal buffer.
5   if  $k \leq |\mathcal{M}|$  then
6      $\mathcal{M}(k) \leftarrow (\mathbf{X}, \mathbf{Y})_k$ ;  $\triangleright$  Add sample before full.
7      $\mathcal{M} \leftarrow \mathcal{M} \cup \mathcal{M}(k)$ ;  $\triangleright$  Update the long-term buffer.
8   else
9      $r \sim \text{Uniform}(1, k)$ ;
10    if  $r \leq |\mathcal{M}|$  then
11       $\mathcal{M}(r) \leftarrow (\mathbf{X}, \mathbf{Y})_k$ ;  $\triangleright$  Update the long-term buffer.
12    end
13  end
14 end

```

of the data stream, $|\mathcal{S}|$, remains unknown. This lack of prior knowledge about $|\mathcal{S}|$ renders commonly used algorithms such as simple random sampling or systematic sampling inapplicable to such an online learning paradigm. To address this challenge, we dynamically update the long-term buffer using reservoir sampling. First, the memory buffer is initialized to an empty set. Before full, every observed sample is sequentially stored in the buffer. After that, the newly encountered sample may replace one of the stored samples in the buffer. As depicted in Algorithm 1 $\forall k > |\mathcal{M}|$, the j^{th} observed sample is selected as the candidate with the probability of $|\mathcal{M}|/k$. Once the j^{th} sample becomes the candidate, one of the previously stored samples will be replaced by the candidate with the probability of $1/|\mathcal{M}|$. Let $\mathcal{M}(i)$ denote the i^{th} sample stored in the memory buffer. The update step is implemented via a conditional replacement

$$\mathcal{M}(r) = \begin{cases} (\mathbf{X}, \mathbf{Y})_k, & \text{if } r \leq |\mathcal{M}|, r \sim \text{Uniform}(1, k) \\ \mathcal{M}(r), & \text{otherwise.} \end{cases} \quad (6)$$

where $r \in \mathbb{Z}^+$ is the randomly generated index. As a result, the probability of being stored in the buffer is $|\mathcal{M}|/|\mathcal{S}|$ for each sample in $\mathcal{S}$.

Different from the long-term buffer that stores previously learned knowledge, the other submodule within the memory buffer, that is, the temporal buffer $\mathcal{M}_{\text{tmp}}$, saves one batch of the last encountered training samples in the interim until the next optimization. These temporarily buffered samples from the current task $\mathcal{T}_c$ serve as reference baselines for similarity computation in the selective memory rehearsal phase.

B. Selective Memory Rehearsal

The CL objective (4) presents the adaptation with data in the current task while retaining learned knowledge from past

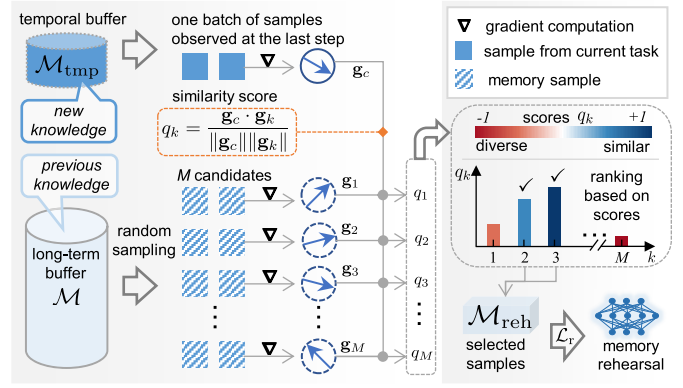


Fig. 3. Selective memory rehearsal for learning plasticity.

tasks. Under assumptions of CL, only the training samples within the current task $\mathcal{T}_c$ and memory buffer $\mathcal{M}$ are available. In such a case, the objective (4) can be further formulated as a constraint optimization problem

$$\begin{aligned} \theta_{1:c}^* = \arg \min_{\theta_{1:c}} \ell(f_{\theta_c}(\mathbf{X}^{\mathcal{D}_{\mathcal{T}_c}}, \mathbf{Y}^{\mathcal{D}_{\mathcal{T}_c}})) \\ \text{s.t. } \ell(f_{\theta_c}, \mathcal{M}) \leq \ell(f_{\theta_{c-1}}, \mathcal{M}) \quad \forall c \in \{1, \dots, N\} \end{aligned} \quad (7)$$

where $\ell(f_{\theta_c}, \mathcal{M})$ denotes the loss on the memory buffer when the model is trained with task $\mathcal{T}_c$. Thus, the constraint could enhance the memory stability by limiting the loss increment on previously learned knowledge. The constraint in (7) can be rephrased in a gradient format

$$\left\langle \frac{\partial \ell(f_{\theta}(\mathbf{X}^{\mathcal{D}_{\mathcal{T}_c}}, \mathbf{Y}^{\mathcal{D}_{\mathcal{T}_c}}))}{\partial \theta}, \frac{\partial \ell(f_{\theta}, \mathcal{M})}{\partial \theta} \right\rangle \geq 0 \quad (8)$$

where $\langle \cdot, \cdot \rangle$ is the inner product function and θ represents all trainable parameters of the full network.

The constraint (8) indicates that using samples from the memory buffer with loss gradients diverse from the current ones might harm the adaptation to task $\mathcal{T}_c$. In other words, the learning plasticity could be impeded by relearning memory samples that are different from training samples $(\mathbf{X}^{\mathcal{D}_{\mathcal{T}_c}}, \mathbf{Y}^{\mathcal{D}_{\mathcal{T}_c}})$. On the other hand, strong learning plasticity means the CL model can utilize learned knowledge from past tasks to enhance its performance in the current task. Motivated by this idea, we unlock the learning plasticity of the motion forecasting model by replaying buffer samples that are similar to the currently learned ones.

Specifically, gradient cosine similarity is employed to score the memory samples, measuring their degree of similarity to the current task. As illustrated in Fig. 3, with the aid of the temporal buffer $\mathcal{M}_{\text{tmp}}$, we compute the loss gradients of all trainable parameters in the full network from the most recently observed sample of the current task $\mathcal{T}_c$. These gradients are then recorded and denoted as $\mathbf{g}_c$. Let B denote the batch size for training. Before the next update step, $M \geq 2B$ samples from the memory buffer $\mathcal{M}$ are randomly selected as candidates for memory rehearsal, and we compute their corresponding loss gradients $\{\mathbf{g}_k\}_{k=1}^M$. Then, we score these M samples by measuring the cosine similarity q_k between $\{\mathbf{g}_k\}_{k=1}^M$ and $\mathbf{g}_c$

$$q_k = \frac{\mathbf{g}_c \cdot \mathbf{g}_k}{\|\mathbf{g}_c\| \|\mathbf{g}_k\|}. \quad (9)$$

The range of the score q_k could vary from -1 to 1 , where the minimum value -1 indicates that the direction of the gradient $\mathbf{g}_k$ is opposite to the gradient of the last observed sample, $\mathbf{g}_c$. In contrast, if the two gradients are the same, the score q_k equals 1 . Since a larger score demonstrates higher similarity between the memory sample and the current task, we rank M candidates based on the scoring results. The B samples that have higher scores are used for the memory rehearsal, denoted as $\mathcal{M}_{\text{reh}}$. Built on these selected memory samples, the proposed selective memory rehearsal is conducted through forward propagation by computing the following loss:

$$\mathcal{L}_r = \mathbb{E}_{(\mathbf{X}, \mathbf{Y}) \sim \mathcal{M}_{\text{reh}}} (\ell(f_\theta(\mathbf{X}), \mathbf{Y})). \quad (10)$$

On the one hand, the selected memory samples have similar gradients to the last encountered samples from the current task, where $\mathcal{M}_{\text{reh}}$ might contain related knowledge for the adaptation to $\mathcal{T}_c$. Thus, the selected memory rehearsal using $\mathcal{M}_{\text{reh}}$ has the potential to improve the performance in the current $\mathcal{T}_c$. It can be considered as a variant of a data augmentation strategy in the context of online CL settings, where the current training data is augmented with a batch of learned samples. On the other hand, unlike using new data to enhance plasticity, the selective memory rehearsal is a type of mild and friendly strategy for memory stability since the reused data originally comes from past tasks. Finally, the total loss is the combination of the loss on current samples $(\mathbf{X}^{\mathcal{T}_c}, \mathbf{Y}^{\mathcal{T}_c})$ and the rehearsal loss (10)

$$\mathcal{L}_{\text{total}} = \ell(f_{\theta_c}(\mathbf{X}^{\mathcal{T}_c}), \mathbf{Y}^{\mathcal{T}_c}) + \mathcal{L}_r. \quad (11)$$

C. Gradient Projection Constraint

The selective memory rehearsal module is inspired by the gradient format goal formulated in (7) and (8), incorporating the rehearsal loss $\mathcal{L}_r$ into the total loss for the enhancement of learning plasticity. Synergistically, the gradient projection constraint module aims at enhancing memory stability. The core idea is to set an inequality constraint for the gradient descent-based optimization on the total loss.

Formally, let $\mathbf{g}$ denote the gradient of the total loss, and let $\mathbf{g}_{\mathcal{M}}$ denote the gradient of the average loss on the long-term buffer. We replace the current loss in (7) with the total loss, where the constraint limits the average loss increment on previously learned knowledge within the long-term buffer $\mathcal{M}$. The optimization with the gradient-format inequality constraint can be represented as

$$\begin{aligned} \theta_{1:c}^* &= \arg \min_{\theta_{1:c}} \mathcal{L}_{\text{total}} \\ \text{s.t. } \langle \mathbf{g}, \mathbf{g}_{\mathcal{M}} \rangle &\geq 0. \end{aligned} \quad (12)$$

As depicted in Fig. 4, when the constraint is satisfied, the motion forecasting model is updated via gradient descent based on $\mathbf{g}$ directly. Otherwise, we project $\mathbf{g}$ to the closest in L2 norm gradient $\tilde{\mathbf{g}}$

$$\begin{aligned} \tilde{\mathbf{g}}^* &= \arg \min_{\tilde{\mathbf{g}}} \frac{1}{2} \|\mathbf{g} - \tilde{\mathbf{g}}\|_2^2 \\ \text{s.t. } \tilde{\mathbf{g}}^\top \mathbf{g}_{\mathcal{M}} &\geq 0. \end{aligned} \quad (13)$$

Since the optimization problem in (13) is a convex quadratic program with a linear inequality constraint, its global optimum can be characterized by the Karush–Kuhn–Tucker (KKT) conditions.

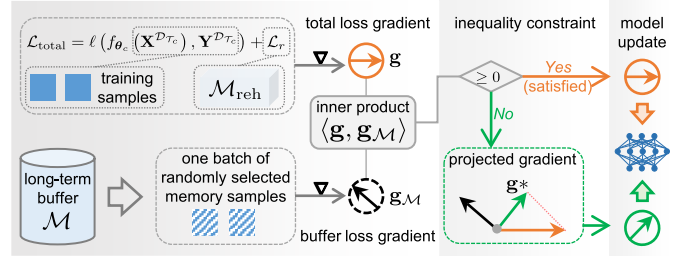


Fig. 4. Gradient projection constraint for memory stability.

The Lagrangian function of (13) is given by

$$L(\tilde{\mathbf{g}}, \lambda) = \frac{1}{2} \|\mathbf{g} - \tilde{\mathbf{g}}\|_2^2 - \lambda \tilde{\mathbf{g}}^\top \mathbf{g}_{\mathcal{M}} \quad (14)$$

where $\lambda \geq 0$ is the Lagrange multiplier.

According to the KKT conditions, the optimal solution $\tilde{\mathbf{g}}^*$ satisfies the stationarity condition

$$\tilde{\mathbf{g}} - \mathbf{g} - \lambda \mathbf{g}_{\mathcal{M}} = \mathbf{0} \quad (15)$$

together with the complementary slackness condition

$$\lambda (\tilde{\mathbf{g}}^\top \mathbf{g}_{\mathcal{M}}) = 0, \quad \lambda \geq 0. \quad (16)$$

From the stationarity condition, we obtain

$$\tilde{\mathbf{g}} = \mathbf{g} + \lambda \mathbf{g}_{\mathcal{M}}. \quad (17)$$

Case 1 (Inactive Constraint): If $\mathbf{g}^\top \mathbf{g}_{\mathcal{M}} \geq 0$, the constraint is inactive, which implies $\lambda = 0$. Therefore, the optimal solution reduces to

$$\tilde{\mathbf{g}}^* = \mathbf{g}. \quad (18)$$

Case 2 (Active Constraint): If $\mathbf{g}^\top \mathbf{g}_{\mathcal{M}} < 0$, the constraint becomes active. According to the complementary slackness condition, we have $\tilde{\mathbf{g}}^\top \mathbf{g}_{\mathcal{M}} = 0$. Substituting the stationarity condition yields

$$\lambda = -\frac{\mathbf{g}^\top \mathbf{g}_{\mathcal{M}}}{\|\mathbf{g}_{\mathcal{M}}\|_2^2}. \quad (19)$$

Accordingly, the projected gradient is given by

$$\tilde{\mathbf{g}}^* = \mathbf{g} - \frac{\mathbf{g}^\top \mathbf{g}_{\mathcal{M}}}{\|\mathbf{g}_{\mathcal{M}}\|_2^2} \mathbf{g}_{\mathcal{M}}. \quad (20)$$

Combining the above two cases, the optimal projected gradient can be summarized as

$$\tilde{\mathbf{g}}^* = \begin{cases} \mathbf{g}, & \text{if } \mathbf{g}^\top \mathbf{g}_{\mathcal{M}} \geq 0 \\ \mathbf{g} - \frac{\mathbf{g}^\top \mathbf{g}_{\mathcal{M}}}{\|\mathbf{g}_{\mathcal{M}}\|_2^2} \mathbf{g}_{\mathcal{M}}, & \text{if } \mathbf{g}^\top \mathbf{g}_{\mathcal{M}} < 0. \end{cases} \quad (21)$$

The projected gradient $\tilde{\mathbf{g}}^*$ is then used to update the model parameters. In implementations, $\mathbf{g}_{\mathcal{M}}$ is computed based on a randomly selected batch of memory samples from the buffer $\mathcal{M}$.

V. EXPERIMENTS

In this section, we first introduce the experimental settings. Second, metrics of motion forecasting and CL are introduced. Then, the proposed SyReM is compared against SOTA CL methods. Furthermore, ablation studies and sensitivity analysis are provided to validate the effectiveness of the key design for SyReM.

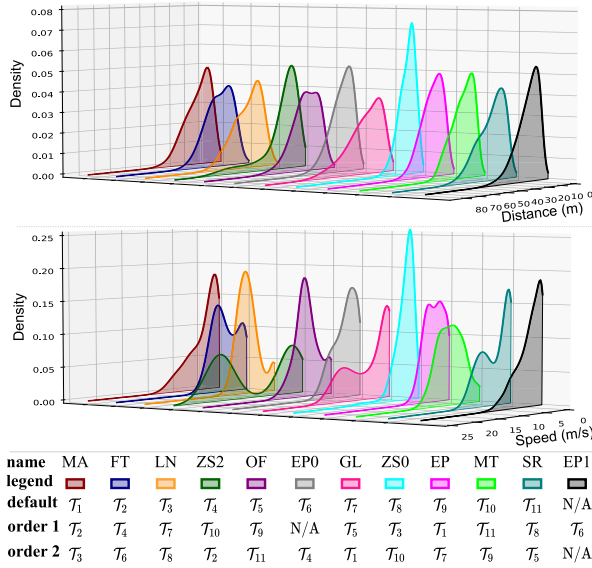


Fig. 5. Feature distributions including the speed (bottom) and the distance between vehicles (top) of scenarios from the INTERACTION dataset [37] used in experiments. Task IDs in the three settings of orders are also demonstrated.

TABLE I

INFORMATION OF SCENARIOS USED IN DIFFERENT TASK ORDERS, WHERE THE SCENARIO NOT USED IN A SPECIFIC ORDER IS DENOTED AS N/A

Scenario name	Video length (min)	Training cases	Test cases	Task ID (default)	Task ID (order 1)	Task ID (order 2)
MA	107.37	33,456	1,178	$\mathcal{T}_1$	$\mathcal{T}_2$	$\mathcal{T}_3$
FT	207.62	66,256	2,452	$\mathcal{T}_2$	$\mathcal{T}_4$	$\mathcal{T}_6$
LN	227.00	3,400	238	$\mathcal{T}_3$	$\mathcal{T}_7$	$\mathcal{T}_8$
ZS2	94.62	15,400	1,043	$\mathcal{T}_4$	$\mathcal{T}_{10}$	$\mathcal{T}_2$
OF	55.04	7,904	568	$\mathcal{T}_5$	$\mathcal{T}_9$	$\mathcal{T}_{11}$
EP0	37.30	9,136	556	$\mathcal{T}_6$	N/A	$\mathcal{T}_4$
GL	259.43	81,640	3,050	$\mathcal{T}_7$	$\mathcal{T}_5$	$\mathcal{T}_1$
ZS0	94.62	35,512	1,084	$\mathcal{T}_8$	$\mathcal{T}_3$	$\mathcal{T}_{10}$
EP	37.30	12,256	433	$\mathcal{T}_9$	$\mathcal{T}_1$	$\mathcal{T}_7$
MT	37.93	5,380	343	$\mathcal{T}_{10}$	$\mathcal{T}_{11}$	$\mathcal{T}_9$
SR	40.09	13,748	552	$\mathcal{T}_{11}$	$\mathcal{T}_8$	$\mathcal{T}_5$
EP1	37.30	10,048	297	N/A	$\mathcal{T}_6$	N/A

A. Experimental Settings

1) *Dataset and Basic Model*: INTERACTION is a widely used dataset for CL in motion forecasting [37]. Twelve sub-datasets collected from different scenarios are used in our experiments to construct sequential tasks for motion forecasting, as shown in Table I. The distributions of speed and distance between vehicles for the used scenarios are demonstrated in Fig. 5 (top) and Fig. 5 (bottom), respectively.

The basic model implemented in the experiments is the uncertainty quantification network (UQnet) [38], [39], which represents the SOTA for vehicle motion forecasting in the open INTERPRET challenge.¹ The SyReM and all compared methods are integrated into the UQnet framework.

2) *Implementation Details*: Table II summarizes the default implementation details used in our main experiments. All models are optimized using Adam. Experiments are conducted

¹INTERPRET: Interaction-dataset-based prediction challenge, single-agent track, organized by ICCV Competition. Available at: <https://challenge.interaction-dataset.com/leader-board/>. Last accessed on January 25th, 2026.

TABLE II
DEFAULT IMPLEMENTATION DETAILS

Category	Parameter	Symbol	Value
Optimization	Learning rate	–	1×10^{-3}
	Batch size	B	8
	Epoch	–	1
	Total iteration	–	35,512
Rehearsal	Buffer size	$ \mathcal{M} $	1,000
	Candidate number	M	16
Forecasting	Observation time	t_{obs}	1 s
	Prediction horizon	t_{pred}	3 s
	Sampled endpoints	W	6

on a Linux-based server equipped with an AMD 9950X CPU and an NVIDIA GeForce RTX 5090 GPU. Each task is trained for one epoch under a one-pass data stream assumption. Since different tasks contain different numbers of training cases, the number of iterations within each task varies accordingly. With a batch size of 8, the cumulative number of iterations across all sequential tasks is 35 512 for the default task order and changed task order 2, and 35 626 for changed task order 1. The averaged results and standard deviations are obtained by three experimental repeats.

B. Baselines

We compare our proposed method with the following baselines to reveal the effectiveness and advantages of SyReM, including both SOTA CL methods and ablation configurations.

1) SOTA Methods:

- 1) *Vanilla*: Original UQnet without implementing any CL methods. It serves as a non-CL baseline, representing the basic capabilities of the DNN-based motion forecasting model in CL tasks.
- 2) *AdaBOP* [40]: An orthogonal-projection-based CL method. It makes a tradeoff between stability and plasticity by confining gradient updates of new data to the orthogonal subspace of learned tasks.
- 3) *FLB* [36]: A CL method that employs bidirectional regularization from historical and newly acquired knowledge to harmonize stability and plasticity. We implement the FLB injected into EWC [28] as it is the configuration applicable for domain incremental learning of the motion forecasting task.
- 4) *DER* [41]: A CL method, which encourages the model to mimic the initial response to memory samples using a distillation-based rehearsal loss function. The memory samples are collected by reservoir sampling from the online data stream.
- 5) *GSM* [14]: A CL method that applies an inequality constraint to losses on memory samples to reduce forgetting. It relies on manually informed task IDs to allocate an equal amount of memory samples for different tasks.
- 6) *SyReM (Ours)*: This is the proposed online CL method. It is designed to effectively handle sequential data and mitigate catastrophic forgetting, enabling DNN-based motion forecasting models to adaptively learn new tasks over time while retaining knowledge from previous ones.

2) Ablation Configurations:

- 1) *V-SR*: This baseline is equivalent to SyReM without the gradient projection module, representing CL strategies that primarily focus on enhancing learning plasticity by rehearsing selected memory samples.
- 2) *V-GP*: The proposed gradient projection constraint module with the long-term buffer to strengthen memory stability. It is equivalent to the proposed method without the selective memory rehearsal module.
- 3) *Sy-Full*: This configuration rehearses all of the samples from the memory buffer at every step. Other settings are the same as the proposed SyReM.
- 4) *U-Sy-U*: In this ablation configuration, the similarity-based memory selection strategy is replaced with a naive random sampling mechanism.
- 5) *U-Sy-UB*: Memory samples that have smaller similarity scores are prioritized for the gradient projection constraint. Other settings are the same as U-Sy-U.
- 6) *B-Sy-B*: The memory buffer is built by prioritizing samples that have smaller similarity scores. Newly encountered samples with gradients more dissimilar to existing memory samples are assigned higher probabilities in this ablation configuration.
- 7) *B-Sy-BB*: Memory samples that have smaller similarity scores are prioritized for the gradient projection constraint. Other settings are the same as B-Sy-B.
- 8) *B-Sy-U*: The memory buffer is the same to B-Sy-B, which maximizes the buffer diversity. The rehearsal strategy is the naive random selection.
- 9) *B-Sy-UB*: Memory samples that have smaller similarity scores are prioritized for the gradient projection constraint. Other settings are the same as B-Sy-U.
- 10) *U-Sy-BB*: It prioritizes memory samples that have smaller similarity scores for the gradient projection. Other settings are the same as SyReM.

C. Evaluation Metrics

1) *Motion Forecasting Metrics*: Minimum final displacement error (FDE) and miss rate (MR) are widely used metrics in motion forecasting [24]. As the basic metrics adopted in this study, the FDE measures the minimum Euclidean distance between the W predicted endpoints and the ground truth $\mathbf{Y}$ over each sample

$$\text{FDE} = \min_{k \in \{1, \dots, W\}} \|\hat{\mathbf{Y}}_k - \mathbf{Y}\|_2. \quad (22)$$

MR measures the percentage of predicted endpoints that are out of a given lateral or longitudinal area of the ground truth. As implemented in [38], the lateral threshold of the given area is 1 m, and the longitudinal threshold is determined by the piecewise function depending on the velocity v of the TA

$$th_{\text{MR}}(v) = \begin{cases} 1, & v < 1.4 \\ 1 + \frac{v - 1.4}{11 - 1.4}, & 1.4 \leq v \leq 11 \\ 2, & v > 11 \end{cases} \quad (23)$$

where the unit of the longitudinal threshold is m, and the unit of the velocity v is ms^{-1} . Let W^{out} denote the number

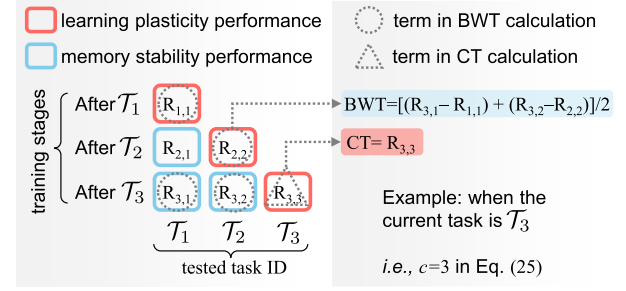


Fig. 6. Schematic of memory stability and learning plasticity metrics. Note that $\text{FDE}_{i,j}$ and $\text{MR}_{i,j}$ are denoted as $R_{i,j}$ uniformly. In this example, the total number N is 3 and the current task T_c incrementally increases from T_1 to T_3 .

of predicted endpoints that are out of the given area within a case. The MR of one test case is calculated as

$$\text{MR} = \frac{W^{\text{out}}}{W} \times 100\%. \quad (24)$$

The FDE and MR within one task are average results of (22) and (24) over the corresponding test set. Since FDE and MR both measure the error, the smaller values of these two metrics represent better performance.

As described in Section III, sequential tasks are constructed to validate the CL performance, where the motion forecasting model is tested in all tasks after learning a new task. Let $\text{FDE}_{i,j}$ and $\text{MR}_{i,j}$ denote the FDE and MR within the test set of task T_j after the model learns the task T_i . The metrics for quantifying the memory stability and learning plasticity are built based on $\text{FDE}_{i,j}$ and $\text{MR}_{i,j}$ in the following.

2) *Memory Stability Metrics*: The FDE-based backward transfer (BWT) and MR-based BWT are used to quantify the memory stability, measuring the influence that learning the current task T_c has on the performance of previously learned tasks $T_i \forall i < c$. We represent the FDE and MR using R , and the FDE-BWT and MR-BWT can be uniformly formulated as

$$\text{R-BWT} = \frac{1}{c-1} \sum_{i=1}^{c-1} (R_{c,i} - R_{i,i}). \quad (25)$$

Note that since the BWT is calculated based on the error metrics (i.e., FDE and MR), the smaller FDE-BWT or MR-BWT represents better memory stability, namely less catastrophic forgetting. In particular, negative values of FDE-BWT or MR-BWT indicate that performance on old tasks improves after learning new tasks, demonstrating beneficial knowledge transfer from new data.

3) *Learning Plasticity Metrics*: The learning plasticity is defined as the ability to acquire new knowledge by training on newly available data [12]. Thus, we adopt the test performance on the current task T_c , which is obtained after sequential training on samples from T_1 to T_c and formulated as $\text{FDE-CT} = \text{FDE}_{c,c}$ and $\text{MR-CT} = \text{MR}_{c,c}$, as the metric to quantify learning plasticity.

As depicted in Fig. 6, an example presents the calculation of the abovementioned metrics. For convenience, the $\text{FDE}_{i,j}$ and $\text{MR}_{i,j}$ are denoted as $R_{i,j}$ uniformly. The horizontal axis refers to the evaluation task ID, and the vertical axis demonstrates the training stages. The FDE-BWT and MR-BWT in (25) are simply denoted as BWT. Similarly, FDE-CT and MR-CT are denoted as CT.

TABLE III

STAGE-WISE COMPARISON OF MEMORY STABILITY AND LEARNING PLASTICITY ACROSS DIFFERENT METHODS (FDE-BASED METRICS)

Stage	Memory Stability (FDE-BWT, ↓)						Learning Plasticity (FDE-CT, ↓)					
	Vanilla	AdaBOP	FLB	DER	GSM	SyReM	Vanilla	AdaBOP	FLB	DER	GSM	SyReM
T_1-T_2	0.40 ± 0.51	0.57 ± 0.33	0.06 ± 0.56	0.45 ± 0.15	-0.20 ± 0.31	-0.09 ± 0.08	1.00 ± 0.07	0.97 ± 0.09	0.97 ± 0.04	0.94 ± 0.03	0.98 ± 0.06	0.90 ± 0.06
T_1-T_3	0.99 ± 0.09	0.81 ± 0.17	0.80 ± 0.17	1.07 ± 0.30	0.20 ± 0.41	0.05 ± 0.10	0.81 ± 0.02	0.85 ± 0.02	1.05 ± 0.20	0.84 ± 0.03	0.90 ± 0.13	0.85 ± 0.03
T_1-T_4	1.37 ± 0.23	1.08 ± 0.16	0.80 ± 0.20	0.45 ± 0.09	0.00 ± 0.03	0.05 ± 0.04	0.71 ± 0.05	0.67 ± 0.04	0.84 ± 0.18	0.74 ± 0.12	0.71 ± 0.06	0.71 ± 0.01
T_1-T_5	1.20 ± 0.06	1.88 ± 0.36	1.05 ± 0.46	0.97 ± 0.76	0.08 ± 0.12	-0.01 ± 0.03	0.96 ± 0.07	1.01 ± 0.06	0.94 ± 0.05	0.96 ± 0.05	0.99 ± 0.09	0.91 ± 0.03
T_1-T_6	1.22 ± 0.06	1.49 ± 0.10	1.61 ± 0.78	0.67 ± 0.20	0.14 ± 0.11	0.58 ± 0.67	0.72 ± 0.02	0.71 ± 0.01	0.79 ± 0.03	0.73 ± 0.02	0.71 ± 0.01	0.72 ± 0.01
T_1-T_7	0.71 ± 0.12	0.71 ± 0.13	0.67 ± 0.20	0.46 ± 0.08	0.20 ± 0.11	0.16 ± 0.05	0.78 ± 0.01	0.75 ± 0.01	0.83 ± 0.04	0.76 ± 0.02	0.71 ± 0.06	0.70 ± 0.06
T_1-T_8	1.96 ± 0.08	1.50 ± 0.05	1.44 ± 0.25	0.44 ± 0.04	0.22 ± 0.11	0.14 ± 0.05	0.60 ± 0.01	0.60 ± 0.01	0.64 ± 0.03	0.59 ± 0.01	0.66 ± 0.06	0.66 ± 0.06
T_1-T_9	1.47 ± 0.18	1.38 ± 0.23	1.50 ± 0.26	0.49 ± 0.14	0.15 ± 0.06	0.14 ± 0.05	0.74 ± 0.02	0.72 ± 0.02	0.77 ± 0.04	0.74 ± 0.01	0.69 ± 0.01	0.73 ± 0.02
T_1-T_{10}	0.48 ± 0.14	0.63 ± 0.06	0.39 ± 0.04	0.34 ± 0.05	0.12 ± 0.04	0.10 ± 0.03	0.74 ± 0.03	0.74 ± 0.06	0.70 ± 0.02	0.68 ± 0.02	0.70 ± 0.09	0.63 ± 0.10
T_1-T_{11}	0.91 ± 0.13	0.63 ± 0.05	0.70 ± 0.09	0.35 ± 0.15	0.12 ± 0.05	0.09 ± 0.02	0.56 ± 0.04	0.55 ± 0.02	0.58 ± 0.03	0.56 ± 0.02	0.59 ± 0.06	0.56 ± 0.09
Avg.	1.07 ± 0.09	1.07 ± 0.02	0.90 ± 0.10	0.57 ± 0.08	0.10 ± 0.11	0.12 ± 0.09	0.76 ± 0.01	0.76 ± 0.02	0.81 ± 0.03	0.75 ± 0.02	0.77 ± 0.01	0.74 ± 0.00

TABLE IV

STAGE-WISE COMPARISON OF MEMORY STABILITY AND LEARNING PLASTICITY ACROSS DIFFERENT METHODS (MR-BASED METRICS)

Stage	Memory Stability (MR-BWT, ↓)						Learning Plasticity (MR-CT, ↓)					
	Vanilla	AdaBOP	FLB	DER	GSM	SyReM	Vanilla	AdaBOP	FLB	DER	GSM	SyReM
T_1-T_2	9.73 ± 15.51	15.39 ± 10.96	-0.93 ± 16.62	9.88 ± 3.37	-7.33 ± 8.08	-3.82 ± 3.11	7.52 ± 2.22	6.23 ± 2.08	6.57 ± 1.07	5.71 ± 0.81	7.07 ± 1.77	4.46 ± 1.02
T_1-T_3	24.07 ± 2.93	21.88 ± 3.51	18.04 ± 2.22	25.81 ± 4.20	4.46 ± 11.51	0.59 ± 2.56	0.84 ± 0.34	1.82 ± 0.40	7.00 ± 5.76	1.68 ± 0.59	3.64 ± 3.67	2.52 ± 0.91
T_1-T_4	35.12 ± 5.14	28.59 ± 5.66	22.23 ± 6.35	13.56 ± 2.95	0.80 ± 1.83	1.11 ± 0.10	0.80 ± 0.20	0.89 ± 0.27	5.43 ± 5.36	3.23 ± 3.56	1.18 ± 0.32	1.21 ± 0.70
T_1-T_5	23.80 ± 1.01	29.82 ± 5.43	19.55 ± 3.52	16.47 ± 11.34	0.91 ± 3.72	-0.76 ± 0.89	4.81 ± 1.22	5.16 ± 0.54	4.64 ± 1.30	5.11 ± 1.90	4.75 ± 1.63	3.46 ± 0.22
T_1-T_6	22.78 ± 1.57	22.15 ± 0.53	26.21 ± 10.56	18.96 ± 6.38	4.46 ± 4.70	5.66 ± 5.26	1.44 ± 0.78	0.90 ± 0.15	1.80 ± 0.59	0.96 ± 0.08	0.84 ± 0.45	0.96 ± 0.34
T_1-T_7	14.71 ± 1.20	16.77 ± 1.61	14.08 ± 1.33	10.43 ± 2.34	5.74 ± 2.47	3.77 ± 1.22	4.81 ± 0.40	3.95 ± 0.35	5.88 ± 1.19	4.58 ± 0.79	2.84 ± 2.04	2.78 ± 2.08
T_1-T_8	34.41 ± 2.32	29.99 ± 1.42	28.44 ± 3.57	11.80 ± 1.08	5.45 ± 2.36	3.19 ± 1.19	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.91 ± 1.16	0.39 ± 0.55
T_1-T_9	18.42 ± 2.61	17.47 ± 3.42	17.38 ± 6.38	10.02 ± 1.40	3.26 ± 1.51	2.40 ± 0.81	1.54 ± 0.29	1.23 ± 0.39	2.08 ± 0.95	1.47 ± 0.58	1.06 ± 0.23	1.08 ± 0.29
T_1-T_{10}	9.06 ± 2.33	12.45 ± 1.67	7.80 ± 0.64	7.91 ± 0.79	2.46 ± 0.95	1.47 ± 0.72	1.26 ± 0.55	0.42 ± 0.23	1.17 ± 0.41	1.17 ± 0.24	0.80 ± 0.33	0.86 ± 0.25
T_1-T_{11}	11.67 ± 0.96	10.72 ± 0.57	9.16 ± 1.33	7.86 ± 2.68	2.07 ± 1.53	1.30 ± 0.51	0.66 ± 0.23	0.42 ± 0.23	0.72 ± 0.15	0.42 ± 0.09	1.21 ± 0.82	0.64 ± 0.27
Avg.	20.38 ± 2.39	20.52 ± 1.52	16.19 ± 3.66	13.27 ± 1.96	2.23 ± 2.86	1.49 ± 1.09	2.37 ± 0.21	2.23 ± 0.36	3.53 ± 0.94	2.43 ± 0.48	2.43 ± 0.51	1.84 ± 0.23

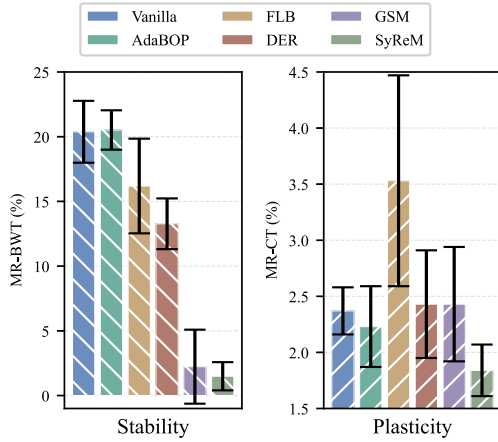


Fig. 7. Averaged memory stability measured by MR-BWT (left) and learning plasticity measured by MR-CT (right) across methods. Lower values indicate better performance, and error bars denote the standard deviation from repeated experiments.

D. Performance Comparison Against SOTA Methods

We compare the proposed SyReM against SOTA CL methods, including AdaBOP [40], FLB [36], DER [41], and GSM [14] in the default task order setting. For every method, the motion forecasting model is incrementally trained by the one-pass online data stream, and then evaluated in all test sets of $\{T_i\}_{i=1}^N$ each time it finishes observing the training samples of one task $T_i \forall i \in [1, N]$.

Memory stability (measured by MR-BWT) and learning plasticity (measured by MR-CT), averaged across 11 tasks, are compared, as depicted in Fig. 7. We find that the compared SOTA CL methods face the stability-plasticity dilemma. Specifically, FLB, DER, GSM, and SyReM have smaller

MR-BWT values than Vanilla (colored in blue), indicating improved memory stability over the non-CL baseline. However, experimental results show reduced learning plasticity in FLB, DER, and GSM with increased MR-CT compared to Vanilla. The significant plasticity degradation occurs in FLB, where its MR-CT value of 3.53 is approximately 48.9% higher than Vanilla's 2.37. While AdaBOP achieves a smaller MR-CT than Vanilla (indicating better learning plasticity), it fails to improve memory stability. Notably, SyReM escapes such a dilemma. It has an averaged MR-BWT of 1.49%, significantly alleviating the catastrophic forgetting of Vanilla with the averaged MR-BWT of 20.38%. On the other hand, Vanilla and AdaBOP are two baselines with strong learning plasticity. SyReM achieves an MR-CT of 1.84%, representing a reduction of 22.4% from Vanilla (2.37) and 17.5% from AdaBOP (2.23), showing a superior adaptation to new tasks.

Comprehensive comparisons with detailed FDE-based and MR-based metrics across sequential training stages are reported in Tables III and IV, respectively.² For memory stability, GSM and SyReM achieve the smallest FDE-BWT and MR-BWT across most learning stages. Negative FDE-BWT and MR-BWT are observed in several cases, indicating performance gains on prior tasks after learning new data. We also observe that replay-based CL methods (including DER, GSM, and SyReM) exhibit advanced stability compared to the remaining baselines. However, the learning plasticity of the non-CL baseline is not enhanced by these replay-based methods. Compared to Vanilla, DER and GSM have similar FDE-CT in most cases, and severely increased MR-CT in specific stages. For example, in the $T_1 - T_4$ stage, DER has an MR-CT of 3.23%, whereas the MR-CT of Vanilla is only

²Standard deviations are reported to two decimal places. Values shown as 0.00 indicate minimal fluctuations (≤ 0.005) from repeated experiments, not absolute zero variance.

TABLE V
ABLATION EXPERIMENTS ON CORE MODULES AND MECHANISMS

Method	Memory stability ($\downarrow$)		Learning plasticity ($\downarrow$)	
	FDE-BWT	MR-BWT	FDE-CT	MR-CT
Vanilla	1.07 ± 0.09	20.38 ± 2.39	0.76 ± 0.01	2.37 ± 0.21
V-GP	0.16 ± 0.12	4.97 ± 2.38	0.76 ± 0.01	2.51 ± 0.53
V-SR	0.14 ± 0.03	3.45 ± 0.59	0.74 ± 0.02	1.93 ± 0.35
Sy-full	0.40 ± 0.05	10.07 ± 1.06	0.86 ± 0.02	4.25 ± 0.03
SyReM	0.12 ± 0.09	1.49 ± 1.09	0.74 ± 0.00	1.84 ± 0.23

0.80%. In the $\mathcal{T}_1 - \mathcal{T}_3$ stage, the MR-CT of Vanilla (0.84%) is approximately 76.92% lower than that of GSM (3.64%).

Different from replay-based methods requiring a memory buffer, AdaBOP and FLB are computationally efficient CL methods. Nevertheless, their memory stability improvements remain limited in our experiments. The BWT values of AdaBOP are comparable to the Vanilla baseline across most training stages. Observed BWT reductions are insufficient for meaningful average stability gains over the non-CL baseline. FLB achieves improved memory stability with an average FDE-BWT of 0.90 m, 15.89% lower than Vanilla's 1.07 m. Yet, the magnitude of its stability enhancement is less pronounced than that of replay-based methods. Our proposed SyReM achieves a much lower average FDE-BWT of 0.12 m, an 88.79% reduction from Vanilla and 86.67% from FLB, showcasing significantly better memory stability. In plasticity evaluations, AdaBOP achieves the lowest FDE-CT among competitors in three stages and a lower average MR-CT than Vanilla, FLB, DER, and GSM, showcasing superior learning plasticity. This may benefit from its adaptive strategy of adjusting optimized gradients to new data. By comparison, FLB incurs the highest FDE-CT and MR-CT in most stages. Its regularization for memory stability retention may impair adaptability to new tasks.

In summary, the compared SOTA methods struggle with the inherent stability-plasticity dilemma. Replay-based methods offer strong memory stability but insufficient plasticity. FLB shows limited stability gains alongside poor adaptability to new tasks, and AdaBOP excels in plasticity yet fails to enhance memory stability. The proposed SyReM achieves the best performance in both metrics across most test cases, demonstrating a superior tradeoff that outperforms these SOTA methods in online CL experiments for motion forecasting.

E. Ablation Study

1) *Ablation of Core Modules:* To investigate the contribution of core modules in SyReM, the CL performance of V-GP and V-SR (measured by averaged metrics in 11 default tasks) is demonstrated as shown in Table V. Compared to Vanilla, V-GP and V-SR both have smaller FDE-BWT and MR-BWT, indicating that applying the proposed selective rehearsal or gradient projection module solely can alleviate catastrophic forgetting. Moreover, V-SR exhibits superior memory stability over V-GP by a marginal margin, as evidenced by its MR-BWT value of 3.45% compared to 4.97% of V-GP. This result shows that selective rehearsal does not impair memory stability but rather enhances it through sample replay. Besides,

TABLE VI
ABLATION PERFORMANCE OF MEMORY STABILITY (FDE-BWT/MR-BWT) AND LEARNING PLASTICITY (FDE-CT/MR-CT)

Method	Buffer	Rehearsal	Projection	Stability ($\downarrow$)	Plasticity ($\downarrow$)
B-Sy-UB	biased	unbiased	biased	0.15 / 3.75	0.80 / 2.79
B-Sy-BB	biased	biased	biased	1.08 / 18.60	0.86 / 4.76
U-Sy-UB	unbiased	unbiased	biased	0.04 / 0.82	0.84 / 4.14
U-Sy-BB	unbiased	biased	biased	0.09 / 2.19	0.78 / 2.73
B-Sy-U	biased	unbiased	unbiased	0.24 / 6.32	0.79 / 2.54
B-Sy-B	biased	biased	unbiased	0.41 / 7.19	0.80 / 3.25
U-Sy-U	unbiased	unbiased	unbiased	0.05 / 1.05	0.81 / 3.60
SyReM	unbiased	biased	unbiased	0.12 / 1.49	0.77 / 2.67

the learning plasticity is enhanced by V-SR, while V-GP yields similar average FDE-CT and MR-CT to Vanilla, showing little contribution to adapting to new tasks. In this ablation study, SyReM achieves the lowest values across all error-based metrics, which validates the synergistic effect of the integrated design in improving both memory stability and learning plasticity.

For a more intuitive demonstration, experimental cases are shown in Fig. 8. Models are sequentially trained on $\mathcal{T}_1$ to $\mathcal{T}_5$. In each subfigure, red and blue rectangles denote TA and SAs, respectively. Green triangles represent TA's ground truth positions, and gold stars with red heatmaps indicate predictions. By comparing $\mathcal{T}_1$ test cases (post- $\mathcal{T}_1$ versus post- $\mathcal{T}_5$ training), catastrophic forgetting is observed: Vanilla's predictions deviate more than 3 m from ground truth after $\mathcal{T}_5$ training, while V-GP and SyReM maintain accurate predictions. The third and fourth columns of Fig. 8 show $\mathcal{T}_3$ and $\mathcal{T}_5$ test cases for adapting to a new task: predictions from V-GP deviate drastically (severe loss of learning plasticity), whereas Vanilla and SyReM yield accurate predictions.

Sy-full is evaluated to verify the efficacy of our selective rehearsal mechanism. This model replays all buffer samples per training step. It improves memory stability over Vanilla, with FDE-BWT reduced by 62.6%, from 1.07 to 0.40 m, and MR-BWT shows similar improvements. However, SyReM still outperforms Sy-full in mitigating catastrophic forgetting. Sy-full exhibits less optimal learning plasticity, as its exhaustive replay strategy exacerbates plasticity loss and leads to higher FDE-CT and MR-CT than Vanilla. Sy-full might intuitively serve as a CL upper bound due to its maximal data usage, yet the results demonstrate that this aggressive replay strategy fails to balance new task learning and knowledge retention effectively.

2) *Ablation of Sampling Strategies:* To investigate the impacts of buffer construction, memory rehearsal, and gradient projection module strategies, we conduct ablation experiments on SyReM by replacing its original strategies with alternative ones. For buffer construction, SyReM uses reservoir sampling (unbiased data stream estimation), while the alternative adopts gradient similarity-based sampling prioritizing diverse samples. For memory rehearsal, we compare selective (similar to the current task) and random batch replay. For the gradient projection module, we contrast random sampling with sampling prioritizing low-similarity-score samples for projection constraint loss calculation. Combinatorial comparisons of these strategies are presented in Table VI.

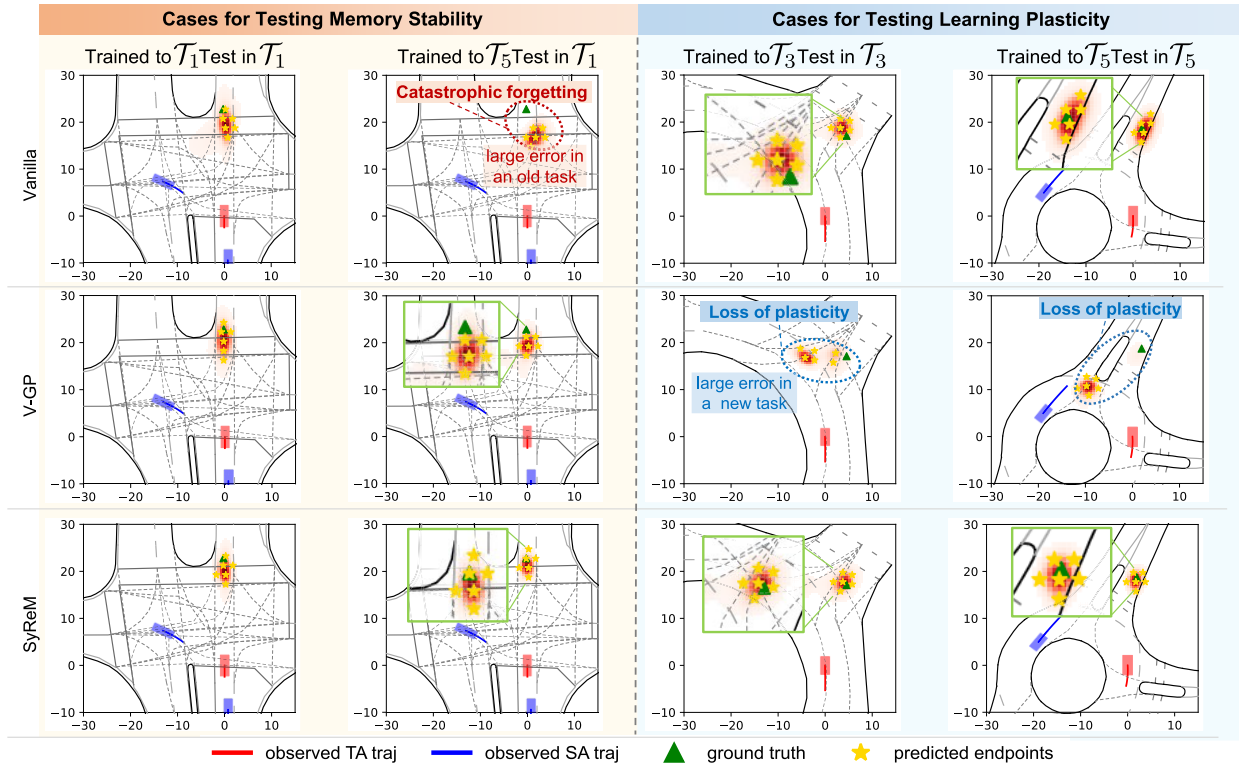


Fig. 8. Experimental cases showing the stability-plasticity dilemma of baseline models, which is addressed by the proposed SyReM.

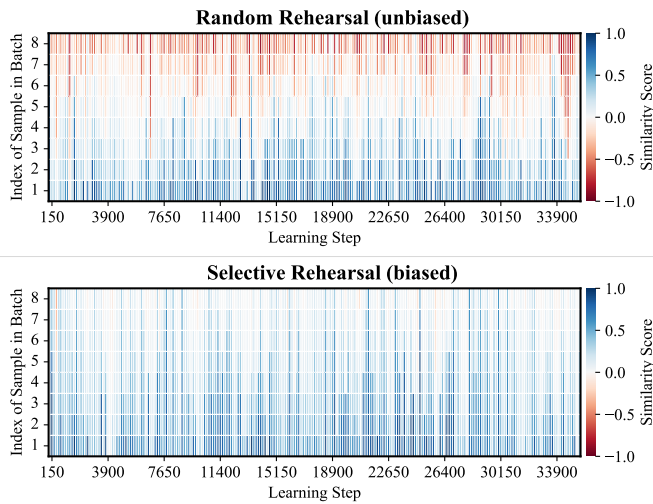


Fig. 9. Similarity scores quantify the similarity between rehearsed memory samples and current task samples, with larger values indicating higher similarity. Random rehearsal (top, U-Sy-U) and selective rehearsal (bottom, SyReM) are illustrated with a downsampling rate of 150.

U-Sy-U and SyReM both construct memory buffers via reservoir sampling, with U-Sy-U replaying random batches and SyReM selectively replaying samples with high gradient cosine similarity to the current task. As shown in Table VI, U-Sy-U achieves superior memory stability with lower FDE-BWT and MR-BWT, while SyReM reduces FDE-CT by 5% (0.77 versus 0.81 m) and MR-CT by 25.8%, demonstrating enhanced plasticity. As depicted in Fig. 9, most samples selected by selective rehearsal have positive cosine similarity

values, while nearly half of randomly replayed samples show negative values. These results indicate that random rehearsal ensures broader coverage of past knowledge, whereas gradient-based selective rehearsal leverages related memory to boost current task performance. B-Sy-B uses the same rehearsal strategy as SyReM but adopts selective sampling for buffer construction, and SyReM achieves lower BWT and CT values than B-Sy-B. However, it is premature to conclude that biased buffer construction is inherently inferior. B-Sy-U and B-Sy-B share the same biased buffer construction strategy, with B-Sy-U using random rehearsal. SyReM and B-Sy-U show comparable learning plasticity, yet SyReM exhibits superior memory stability.

Building on the above analysis of buffer construction and memory rehearsal strategies, we further study ablation results of sampling strategies for the gradient projection module. Comparing B-Sy-UB, B-Sy-BB, U-Sy-UB, and U-Sy-BB, unbiased buffer construction delivers stronger stability, which aligns with observations under random gradient projection sampling. U-Sy-UB achieves the best stability with FDE-BWT 0.04 m and MR-BWT 0.82%, while its plasticity is slightly higher than U-Sy-U and weaker than SyReM. SyReM and U-Sy-BB show highly similar performance. B-Sy-BB with fully biased sampling strategies in all modules performs significantly worse than SyReM. It achieves an FDE-BWT of 1.08 m, nearly nine times higher than SyReM's 0.12 m, and an MR-CT of 4.76%, approximately 1.78 times higher than SyReM's 2.67%, demonstrating the critical importance of unbiased random sampling in CL.

These experimental results indicate that the effectiveness of memory rehearsal and gradient projection is dependent on buffer construction. Our empirical results suggest that

TABLE VII

HYPERPARAMETERS SENSITIVITY. STABILITY AND PLASTICITY ARE REPORTED AS FDE-BWT/MR-BWT AND FDE-CT/MR-CT

Buffer size $ \mathcal{M} $	Candidates M	Stability ($\downarrow$)	Plasticity ($\downarrow$)
500	16	0.04 / 0.10	0.80 / 3.54
1,000	16	0.12 / 1.49	0.77 / 2.67
2,000	16	0.10 / 2.12	0.79 / 2.77
1,000	8	0.23 / 1.93	0.78 / 2.84
1,000	16	0.12 / 1.49	0.77 / 2.67
1,000	32	0.09 / 1.48	0.80 / 3.49

under limited memory capacity, unbiased buffer construction (via reservoir sampling) is a suitable configuration for motion forecasting in CL tasks, and pairing it with selective rehearsal and random gradient projection yields a favorable stability–plasticity tradeoff.

F. Sensitivity Analysis

1) *Hyperparameter Sensitivity Experiments*: We perform a sensitivity analysis on buffer size $|\mathcal{M}|$ and candidate number M . As demonstrated in Table VII, for the ablation experiments on varying buffer sizes (fixed $M = 16$), FDE-BWT fluctuates between 0.04 and 0.12 m, while MR-BWT increases from 0.10% to 2.12% as buffer size grows. The best learning plasticity is at $|\mathcal{M}| = 1000$ (FDE-CT = 0.77 m, MR-CT = 2.67%). Larger buffers (e.g., $|\mathcal{M}| = 2000$) offer no consistent stability gains and incur higher storage costs. We empirically suggest that our default buffer size (i.e., 1000) could be a practical choice.

For candidate number (fixed $|\mathcal{M}| = 1000$), increasing M from 8 to 32 improves memory stability. For example, MR-BWT is reduced from 1.93% to 1.48%. However, comparing the settings of $M = 16$ and $M = 32$, increasing M does not help improve the learning plasticity. Meanwhile, a larger M also increases computational overhead for sample ranking. Thus, $M = 16$ could be an empirically suggested configuration for the stability–plasticity tradeoff.

2) *Influence of Task Orders*: In real-world motion forecasting scenarios, the sequence of traffic scenarios is inherently stochastic and unpredictable. To address this concern and validate the superiority of SyReM under varying task orders, we conducted additional experiments by shuffling the task sequence. The corresponding task ID information of the altered orders is presented in Table I.

We find that AdaBOP and FLB perform comparably to Vanilla across the two changed task orders, as shown in Table VIII. They only achieve moderate improvements in learning plasticity over Vanilla under changed task order 2, and their favorable performance under the default task order is not retained, indicating that these buffer-free methods are sensitive to task order. Furthermore, replay-based methods, DER and GSM, yield substantial improvements in Vanilla’s memory stability across both settings. GSM and DER exhibit notable negative BWT values in changed task order 1, demonstrating that newly learned knowledge enhances their performance on old tasks. Nevertheless, DER and GSM fail to balance stability and plasticity effectively, with higher FDE-CT and MR-CT values than Vanilla. For instance, DER and GSM have average

TABLE VIII

SENSITIVITY TO TASK ORDERS. STABILITY AND PLASTICITY ARE REPORTED AS FDE-BWT/MR-BWT AND FDE-CT/MR-CT

Method	Changed Task Order 1		Changed Task Order 2	
	stability ($\downarrow$)	plasticity ($\downarrow$)	stability ($\downarrow$)	plasticity ($\downarrow$)
Vanilla	0.45 / 10.16	0.79 / 3.39	1.17 / 15.31	0.79 / 2.96
AdaBOP	0.36 / 10.23	0.82 / 3.87	1.66 / 18.63	0.78 / 2.73
FLB	0.42 / 10.20	0.81 / 4.24	1.12 / 15.71	0.79 / 2.78
DER	-0.07 / 0.74	1.00 / 7.34	0.41 / 9.07	0.76 / 3.01
GSM	-0.48 / -9.11	1.07 / 8.26	0.20 / 4.10	0.80 / 3.90
SyReM	0.11 / 2.46	0.76 / 2.26	0.09 / 1.90	0.77 / 2.35

TABLE IX

GENERALIZATION TO UNSEEN TASKS MEASURED BY FORWARD TRANSFER: FDE-FWT(M)/MR-FWT(%)

Stage	Vanilla	AdaBOP	FLB	DER	GSM	SyReM
T_1	2.50 / 35.00	2.25 / 29.00	3.14 / 43.82	2.64 / 33.09	2.95 / 39.54	2.21 / 31.74
T_1-T_2	1.83 / 15.54	1.76 / 14.33	1.89 / 16.01	1.74 / 13.62	1.67 / 14.41	1.40 / 14.47
T_1-T_3	2.36 / 20.36	2.25 / 18.88	2.38 / 21.80	2.34 / 19.68	2.10 / 16.12	1.48 / 12.98
T_1-T_4	1.66 / 21.59	1.55 / 18.26	1.51 / 20.30	1.25 / 14.91	1.05 / 9.75	1.02 / 7.91
T_1-T_5	1.00 / 7.59	1.09 / 10.59	1.04 / 9.58	0.99 / 8.01	0.85 / 4.16	0.85 / 4.46
T_1-T_6	0.89 / 5.98	0.90 / 6.72	1.18 / 13.12	0.96 / 9.43	0.88 / 6.36	0.83 / 3.45
T_1-T_7	0.79 / 3.70	0.83 / 3.96	0.87 / 4.36	0.80 / 3.97	0.75 / 1.99	0.77 / 2.71
T_1-T_8	1.14 / 12.18	1.01 / 9.88	1.03 / 10.12	0.82 / 4.48	0.74 / 2.82	0.73 / 2.64
T_1-T_9	1.15 / 13.63	0.97 / 10.50	1.36 / 16.98	0.81 / 5.24	0.70 / 2.22	0.72 / 1.53
T_1-T_{10}	0.58 / 1.03	0.59 / 1.45	0.59 / 1.45	0.59 / 0.97	0.66 / 2.14	0.68 / 1.79
Avg.	1.39 / 13.66	1.32 / 12.36	1.50 / 15.75	1.29 / 11.34	1.23 / 9.95	1.07 / 8.37

MR-CT values of 7.34% and 8.26%, which are 116.52% and 143.66% higher than Vanilla’s 3.39%, respectively, reflecting impaired learning plasticity. In changed task order 2, DER and GSM still mitigate forgetting in the non-CL model with lower BWT, yet their CT values are comparable to or higher than Vanilla.

The distinct advantage of SyReM lies in enhanced learning plasticity while maintaining favorable memory stability. In changed task order 1, SyReM achieves the optimal learning plasticity with an FDE-CT of 0.76 m and MR-CT of 2.26%. In changed task order 2, SyReM attains the best stability among all methods and reduces MR-CT by 13.92% and 15.47% compared with AdaBOP (2.73) and FLB (2.78), respectively. Collectively, these results demonstrate that SyReM consistently delivers a favorable stability–plasticity tradeoff, validating its robustness across varying task sequences.

G. Generalization Beyond the Dilemma

Leveraging learned knowledge to enhance performance in totally unseen tasks is also important in the online CL setting when the data is unavailable in new tasks [11]. Forward transfer (FWT) is used to measure the zero-shot generalization. We use R to represent FDE and MR uniformly $\forall c \leq N - 1$

$$\text{R-FWT} = \frac{1}{N - c} \sum_{k=c+1}^N R_{c,k} \quad (26)$$

where N is the total number of CL tasks, and FDE-FWT and MR-FWT evaluate the average performance on unseen tasks $\{\mathcal{T}_k\}_{k=c+1}^N$ after sequentially learned tasks $\{\mathcal{T}_i\}_{i=1}^c$. The smaller FDE-FWT and MR-FWT indicate better zero-shot generalization.

Experimental results of FDE-FWT and MR-FWT in different stages under the default task order are shown in Table IX. The proposed SyReM has smaller values of FDE-FWT and

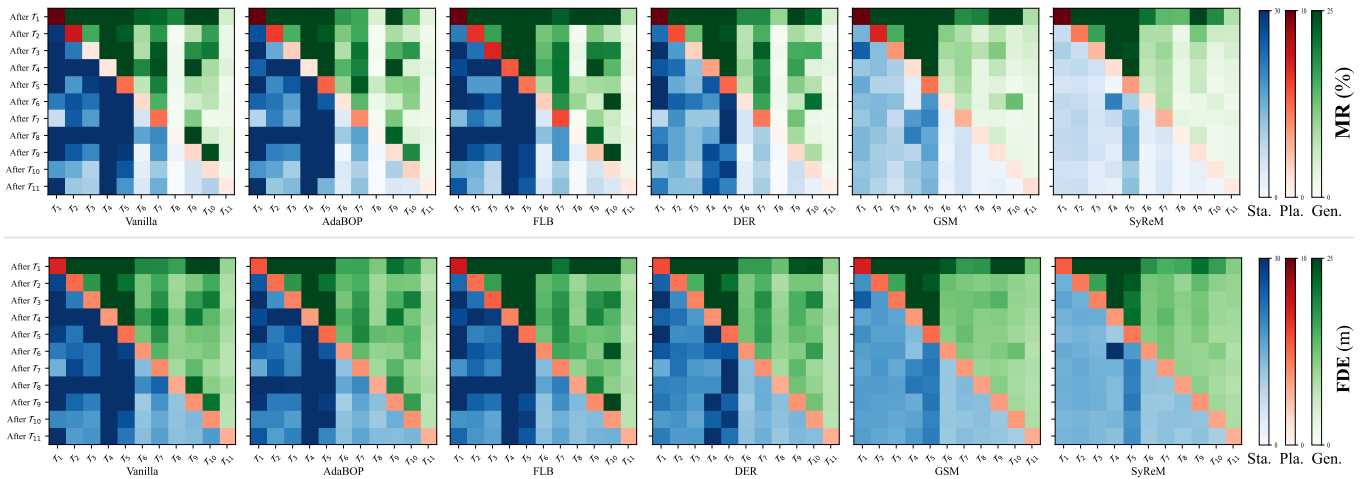


Fig. 10. Detailed FDE (m) and MR (%) at each training stage. Within each subfigure (array), the horizontal axis refers to the testing task ID, and the vertical axis represents different CL stages. The blue colored areas reflect the performance on learned tasks (stability). The red colored ones are performance on each current task (plasticity), and the green ones are for generalization in unseen tasks.

TABLE X

TRAINING AND INFERENCE TIMES ARE REPORTED AS AVERAGE TIME PER BATCH, WHILE PEAK GPU MEMORY IS MEASURED IN MB

Method	Train time (ms)	Total train time (s)	Peak mem. (MB)	Infer time (ms)
Vanilla	36.85	1309	2958.21	9.16 ± 0.43
AdaBOP	37.57	1334	2958.21	9.37 ± 0.11
FLB	76.86	2729	2994.07	9.15 ± 0.10
DER	83.93	2981	5959.93	9.22 ± 0.43
GSM	62.91	2233	2967.47	9.30 ± 0.42
Sy-full	2658.86	94415	3035.78	9.11 ± 0.30
SyReM	344.44	12231	5813.63	9.10 ± 0.39

MR-FWT than the compared SOTA baselines in most cases, showing a better generalization in unseen tasks. Moreover, Fig. 10 illustrates the detailed FDE and MR values of the compared methods among previously learned tasks (colored in blue tones), current tasks (red tones), and unseen tasks (green tones) at each training stage. Lighter color shades represent smaller FDE or MR values, indicating better performance. Compared with baselines, SyReM exhibits relatively lighter-colored regions in both red and blue tones, showing superior stability and plasticity. Besides, most CL baselines and SyReM outperform Vanilla on unseen tasks after learning T_5 . These results indicate that the retention of knowledge from historical tasks is beneficial for improving performance in unknown distributions.

H. Computational Cost

The computational cost, including per-batch training time, total training time (for 11 tasks in the default order setting), peak GPU memory, and inference time for motion forecasting, is reported as shown in Table X. AdaBOP and FLB have similar training time and peak GPU memory usage to Vanilla, which are lower than those of the replay-based methods. Compared to baselines, SyReM incurs higher per-batch training time (344.44 ms) and peak GPU memory, yet its overhead is notably lower than Sy-full (2658.86 ms). The additional cost primarily arises from replay-based optimization, where an extra replay batch is sampled from the memory buffer at

each iteration for the selective memory rehearsal module and the gradient projection constraint module. Nevertheless, these mechanisms are essential for achieving the significant stability-plasticity tradeoff improvements observed in CL performance.

Notably, SyReM achieves competitive inference efficiency with an inference time of 9.10 ms, on par with all baselines and well below the 20 ms real-time threshold for AD applications. This advantage arises because our method is plug-and-play for motion forecasting models and does not cause a substantial increase in the neural network scale, avoiding extra inference overhead.

I. Discussion

Comprehensive evaluations of stability, plasticity, and generalization are conducted in our experiments. Replay-based methods, including DER and GSM, provide stronger memory stability improvements for the non-CL model than regularization (e.g., FLB) and orthogonal-projection methods (e.g., AdaBOP) without memory data reuse, yet they exhibit weaker learning plasticity than Vanilla. While AdaBOP demonstrates strong adaptation to new tasks, it exhibits insufficient memory stability. These experimental results show that the compared SOTA CL methods are trapped in the stability-plasticity dilemma. By contrast, SyReM simultaneously enhances memory stability and learning plasticity. Our ablation and sensitivity analyses further reveal that selective memory rehearsal benefits both stability and plasticity, and SyReM exhibits strong robustness under different hyperparameters and task orders. Beyond this dilemma, most CL methods achieve smaller FWT than Vanilla, which indicates that knowledge retrieval improves model generalization in unseen tasks.

VI. CONCLUSION

This study investigates the stability-plasticity dilemma in online CL for motion forecasting. The proposed SyReM escapes the dilemma by simultaneously mitigating catastrophic forgetting and achieving accurate motion forecasting

on new tasks. In the synergetic mechanisms of SyReM, the gradient projection-based inequality constraint ensures memory stability, and the selective memory rehearsal strategy enhances learning plasticity. These findings demonstrate that DNN-based models can leverage prior knowledge to adapt continually to evolving online data distributions, laying a foundation for deploying such models in dynamic real-world scenarios where training samples arrive incrementally. Beyond the dilemma, experimental results further show that SyReM outperforms baselines in zero-shot generalization across unseen tasks, with its modest yet superior improvements highlighting the potential of integrating memory rehearsal with generalization-oriented designs. Future work will explore few-shot or zero-shot learning for DNN-based intelligent systems in unseen domains while preserving incremental knowledge accumulation, aiming to strengthen generalization without sacrificing the stability–plasticity balance.

REFERENCES

- [1] R. Li, S. Katsigiannis, T.-K. Kim, and H. P. H. Shum, “BP-SGCN: Behavioral pseudo-label informed sparse graph convolution network for pedestrian and heterogeneous trajectory prediction,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 8, pp. 1–15, Aug. 2025.
- [2] Q. Liu, X. Li, S. Yuan, and Z. Li, “Decision-making technology for autonomous vehicles: Learning-based methods, applications and future outlook,” in *Proc. IEEE Int. Intell. Transp. Syst. Conf. (ITSC)*, Sep. 2021, pp. 30–37.
- [3] L. Wei, Z. Li, J. Gong, C. Gong, and J. Li, “Autonomous driving strategies at intersections: Scenarios, state-of-the-art, and future outlooks,” in *Proc. IEEE Int. Intell. Transp. Syst. Conf. (ITSC)*, Sep. 2021, pp. 44–51.
- [4] D. Kudithipudi et al., “Biological underpinnings for lifelong learning machines,” *Nature Mach. Intell.*, vol. 4, no. 3, pp. 196–210, 2022.
- [5] S. Mozaffari, O. Y. Al-Jarrah, M. Dianati, P. Jennings, and A. Mouzakitis, “Deep learning-based vehicle behavior prediction for autonomous driving applications: A review,” *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 1, pp. 33–47, Jan. 2022.
- [6] T. Lesort, V. Lomonaco, A. Stoian, D. Maltoni, D. Filliat, and N. Díaz-Rodríguez, “Continual learning for robotics: Definition, framework, learning strategies, opportunities and challenges,” *Inf. Fusion*, vol. 58, pp. 52–68, Jun. 2020.
- [7] Z. Li et al., “Complementary learning system empowers online continual learning of vehicle motion forecasting in smart cities,” 2025, *arXiv:2508.19597*.
- [8] Z. Li et al., “Continual driver behaviour learning for connected vehicles and intelligent transportation systems: Framework, survey and challenges,” *Green Energy Intell. Transp.*, vol. 2, no. 4, Aug. 2023, Art. no. 100103.
- [9] H. Yang, Y. Zhou, J. Wu, H. Liu, L. Yang, and C. Lv, “Human-guided continual learning for personalized decision-making of autonomous driving,” *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 4, pp. 5435–5447, Apr. 2025.
- [10] M. McCloskey and N. J. Cohen, “Catastrophic interference in connectionist networks: The sequential learning problem,” in *Psychology of Learning and Motivation*, vol. 24. Amsterdam, The Netherlands: Elsevier, 1989, pp. 109–165.
- [11] L. Wang, X. Zhang, H. Su, and J. Zhu, “A comprehensive survey of continual learning: Theory, method and application,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 8, pp. 5362–5383, Aug. 2024.
- [12] C. Lyle, Z. Zheng, E. Nikishin, B. Á. Pires, R. Pascanu, and W. Dabney, “Understanding plasticity in neural networks,” in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 23190–23211.
- [13] H. Ma, Y. Sun, J. Li, M. Tomizuka, and C. Choi, “Continual multi-agent interaction behavior prediction with conditional generative memory,” *IEEE Robot. Autom. Lett.*, vol. 6, no. 4, pp. 8410–8417, Oct. 2021.
- [14] Y. Lin, Z. Li, C. Gong, C. Lu, X. Wang, and J. Gong, “Continual interactive behavior learning with traffic divergence measurement: A dynamic gradient scenario memory approach,” *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 3, pp. 2355–2372, Mar. 2024.
- [15] Y. Lin et al., “H2C: Hippocampal circuit-inspired continual learning for lifelong trajectory prediction in autonomous driving,” *IEEE Trans. Intell. Transp. Syst.*, early access, Apr. 14, 2025, doi: [10.1109/TITS.2026.3679635](https://doi.org/10.1109/TITS.2026.3679635).
- [16] P. Bao, Z. Chen, J. Wang, D. Dai, and H. Zhao, “Lifelong vehicle trajectory prediction framework based on generative replay,” *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 12, pp. 13729–13741, Dec. 2023.
- [17] X. Nie, Z. Deng, M. He, M. Fan, and Z. Tang, “Online active continual learning for robotic lifelong object recognition,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 12, pp. 17790–17804, Dec. 2024.
- [18] M. Li, S. Chen, Y. Shen, G. Liu, I. W. Tsang, and Y. Zhang, “Online multi-agent forecasting with interpretable collaborative graph neural networks,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 4, pp. 4768–4782, Apr. 2024.
- [19] Y. Lin, Z. Li, C. Gong, Q. Liu, C. Lu, and J. Gong, “Rethinking trajectory prediction in real-world applications: An online task-free continual learning perspective,” in *Proc. IEEE 26th Int. Conf. Intell. Transp. Syst. (ITSC)*, Sep. 2023, pp. 5020–5026.
- [20] S. Dohare, J. F. Hernandez-Garcia, Q. Lan, P. Rahman, A. R. Mahmood, and R. S. Sutton, “Loss of plasticity in deep continual learning,” *Nature*, vol. 632, no. 8026, pp. 768–774, Aug. 2024.
- [21] H. Xue, D. Q. Huynh, and M. Reynolds, “PoPPL: Pedestrian trajectory prediction by LSTM with automatic route class clustering,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 77–90, Jan. 2021.
- [22] Z. Li et al., “An ensemble learning framework for vehicle trajectory prediction in interactive scenarios,” in *Proc. IEEE Intell. Vehicles Symp. (IV)*, Jun. 2022, pp. 51–57.
- [23] Z. Li, C. Lu, Y. Yi, and J. Gong, “A hierarchical framework for interactive behaviour prediction of heterogeneous traffic participants based on graph neural network,” *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 7, pp. 9102–9114, Jul. 2022.
- [24] D. Li, Q. Zhang, S. Lu, Y. Pan, and D. Zhao, “Conditional goal-oriented trajectory prediction for interacting vehicles,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 12, pp. 18758–18770, Dec. 2024.
- [25] R. Liang, Y. Li, J. Zhou, and X. Li, “STGlow: A flow-based generative framework with dual-graphormer for pedestrian trajectory prediction,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 11, pp. 16504–16517, Nov. 2024.
- [26] Y. Fu, Q. Yan, L. Wang, K. Li, and R. Liao, “MoFlow: One-step flow matching for human trajectory forecasting via implicit maximum likelihood estimation based distillation,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2025, pp. 17282–17293.
- [27] X. Chen, L. Zeng, M. Gao, C. Ding, and Y. Bian, “DiffWT: Diffusion-based pedestrian trajectory prediction with time-frequency wavelet transform,” *IEEE Internet Things J.*, vol. 12, no. 5, pp. 5109–5121, Mar. 2025.
- [28] J. Kirkpatrick et al., “Overcoming catastrophic forgetting in neural networks,” *Proc. Nat. Acad. Sci. USA*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [29] F. Feng, L. Hou, Q. She, R. H. M. Chan, and J. T. Kwok, “Power law in deep neural networks: Sparse network generation and continual learning with preferential attachment,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 7, pp. 8999–9013, Jul. 2024.
- [30] L. Wang, B. Lei, Q. Li, H. Su, J. Zhu, and Y. Zhong, “Triple-memory networks: A brain-inspired method for continual learning,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 5, pp. 1925–1934, May 2022.
- [31] H. Lin, S. Feng, B. Zhang, X. Li, and Y. Ye, “HPCR: Holistic proxy-based contrastive replay for online continual learning,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 8, pp. 14706–14720, Aug. 2025.
- [32] S. A. H. Minoofam, A. Bastanfard, and M. R. Keyvanpour, “TRCLA: A transfer learning approach to reduce negative transfer for cellular learning automata,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 5, pp. 2480–2489, May 2023.
- [33] Z. Zhang, Z. Wu, H. Zhang, and J. Wang, “Meta-learning-based deep reinforcement learning for multiobjective optimization problems,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 10, pp. 7978–7991, Oct. 2023.
- [34] G. Ma, R. Jiang, L. Wang, and H. Tang, “Dual memory model for experience-once task-incremental lifelong learning,” *Neural Netw.*, vol. 166, pp. 174–187, Sep. 2023.
- [35] F. Sarfraz, E. Arani, and B. Zonooz, “Sparse coding in a dual memory system for lifelong learning,” in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 8, pp. 9714–9722.
- [36] L. Mahmoodi, P. Moghadam, M. Hayat, C. Simon, and M. Harandi, “Flashbacks to harmonize stability and plasticity in continual learning,” *Neural Netw.*, vol. 190, Oct. 2025, Art. no. 107616.
- [37] W. Zhan, “Interaction dataset: An international, adversarial and cooperative motion dataset in interactive driving scenarios with semantic maps,” 2019, *arXiv:1910.03088*.

- [38] G. Li, Z. Li, V. L. Knoop, and H. van Lint, "Unravelling uncertainty in trajectory prediction using a non-parametric approach," *Transp. Res. C, Emerg. Technol.*, vol. 163, Jun. 2024, Art. no. 104659.
- [39] G. Li, Z. Li, V. L. Knoop, and J. W. C. V. Lint, "How far ahead should autonomous vehicles start resolving predicted conflicts? Exploring uncertainty-based safety-efficiency trade-off," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 10, pp. 14183–14195, Oct. 2024.
- [40] D. Cheng, Y. Hu, N. Wang, D. Zhang, and X. Gao, "Achieving plasticity-stability trade-off in continual learning through adaptive orthogonal projection," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 35, no. 8, pp. 7485–7498, Aug. 2025.
- [41] M. Boschini, L. Bonicelli, P. Buzzega, A. Porrello, and S. Calderara, "Class-incremental continual learning into the eXtended DER-verse," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 5, pp. 5497–5512, May 2023.



Yunlong Lin received the B.S. degree in mechanical engineering from Beijing Institute of Technology (BIT), Beijing, China, in 2022, where he is currently pursuing the Ph.D. degree.

He is also a Visiting Researcher with the Cyber-Physical Systems Group, the School of Computation, Information and Technology, Technical University of Munich (TUM), Garching, Germany. His research interests include scene understanding, world models, lifelong machine learning, and decision-making for intelligent vehicles and embodied AI.



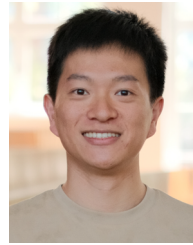
Chao Lu (Member, IEEE) received the B.S. degree in transport engineering from Beijing Institute of Technology (BIT), Beijing, China, in 2009, and the Ph.D. degree in transport studies from the University of Leeds, Leeds, U.K., in 2015.

In 2017, he was a Visiting Researcher with the Advanced Vehicle Engineering Centre, Cranfield University, Cranfield, U.K. He is currently an Associate Professor with the School of Mechanical Engineering, BIT. His research interests include intelligent transportation and vehicular systems, driver behavior modeling, reinforcement learning, and transfer learning and its applications.



Tongshuai Wu received the B.S. degree in vehicle engineering from Beijing Institute of Technology, Beijing, China, in 2025, where he is currently pursuing the M.S. degree with the School of Mechanical Engineering.

His research interests include human–computer interaction, cognitive driving, and path planning of intelligent vehicles.



Xiaocong Zhao (Member, IEEE) received the Ph.D. degree in transportation engineering from Tongji University, Shanghai, China.

From November 2022 to October 2023, he was a Visiting Researcher with the Chair of Traffic Process Automation, Faculty of Transportation and Traffic Sciences "Friedrich List," Technische Universität Dresden, Dresden, Germany. He is currently a Post-Doctoral Research Fellow with the College of Transportation, Tongji University. His research interests include human–machine interaction, social driving behaviors modeling, and interactive decision-making.



Guodong Du received the B.S. degree in mechanical engineering from Beijing Institute of Technology, Beijing, China, in 2019, and the Ph.D. degree in automobile engineering from Beijing Institute of Technology, Beijing, in 2025.

His research interests include motion planning and control, reinforcement learning algorithm, and energy management of hybrid electric vehicles.



Yanwei Sun received the B.Eng. and M.A.Eng. degrees in mechatronic engineering from Jilin University, Changchun, China, in 2001 and 2004, respectively.

His research interests include the technology of wire-controlled chassis, assisted driving systems for intelligent vehicles, and the overall design technology of vehicle platforms.



Zirui Li (Member, IEEE) received the B.S. and Ph.D. degrees in mechanical engineering from Beijing Institute of Technology (BIT), Beijing, China, in 2019 and 2025, respectively.

From June 2021 to July 2022, he was a Visiting Researcher with Delft University of Technology (TU Delft), Delft, The Netherlands. From August 2022 to June 2024, he was a Visiting Researcher with the Chair of Traffic Process Automation, Faculty of Transportation and Traffic Sciences "Friedrich List," TU Dresden, Dresden, Germany. He is currently a Post-Doctoral Research Fellow with Nanyang Technological University, Singapore. His research interests include interactive behavior modeling, risk assessment, and motion planning of automated vehicles.



Jianwei Gong (Member, IEEE) received the B.S. degree from the National University of Défense Technology, Changsha, China, in 1992, and the Ph.D. degree from Beijing Institute of Technology (BIT), Beijing, China, in 2002.

From 2011 to 2012, he was a Visiting Scientist with the Robotic Mobility Group, Massachusetts Institute of Technology, Cambridge, MA, USA. He is currently a Professor with the School of Mechanical Engineering, BIT. His research interests include intelligent vehicle environment perception and understanding, decision making, path/motion planning, and control.